

SYSTEM DESIGN REFERENCE

Databases for System Design

A comprehensive, book-length reference covering every database type, distributed systems theory, indexing internals, caching patterns, and real-world system design — written for senior engineers and technical interviews.

SQL / NoSQL

CAP Theorem

Replication

Sharding

Indexing

Caching

Distributed Systems

System Design Patterns

Back-of-Envelope

Databases for System Design: A Comprehensive Reference

A book-length guide covering every database type, distributed systems theory, and system design patterns — written for engineers preparing for senior-level interviews and building production systems.

Table of Contents

1. [Why Databases Are the Core of System Design](#)
 2. [Relational Databases \(SQL / RDBMS\)](#)
 3. [Document Databases](#)
 4. [Key-Value Stores](#)
 5. [Wide-Column Databases](#)
 6. [Graph Databases](#)
 7. [Time-Series Databases](#)
 8. [Search and Full-Text Engines](#)
 9. [Analytical Databases \(OLAP / Data Warehouses\)](#)
 10. [Vector / Embedding Databases](#)
 11. [NewSQL / Distributed SQL](#)
 12. [Object and Blob Storage](#)
 13. [CAP Theorem, PACELC, and Distributed Systems Theory](#)
 14. [ACID vs BASE — Consistency Models in Depth](#)
 15. [Replication — Internals and Patterns](#)
 16. [Sharding and Partitioning — Strategies and Tradeoffs](#)
 17. [Indexing — Deep Internals](#)
 18. [Caching — Patterns, Strategies, and Pitfalls](#)
 19. [Consistency Patterns and Distributed Coordination](#)
 20. [System Design Patterns Per Use Case](#)
 21. [Numbers Every Engineer Must Know](#)
 22. [Interview Strategy and Common Mistakes](#)
-

Chapter 1: Why Databases Are the Core of System Design

When you are asked to design a system — whether it's Twitter, Uber, Netflix, or a payment processor — the most consequential decisions you will make are about data. Not about which framework to use, not about which cloud provider to pick, and not about microservices vs monolith. The decisions that define the system's correctness, scalability, performance, and operational complexity are almost always about how and where data is stored, replicated, queried, and kept consistent.

Every other component in a system design — load balancers, application servers, message queues, CDNs — is stateless or near-stateless. They can be spun up, killed, and replaced freely. Databases are the opposite. They are the single component that holds state, that defines consistency boundaries, and that will become the bottleneck in nearly every large system. Getting the database choice wrong has consequences that last years: a schema migration on a billion-row table, moving from an eventual-consistency model to a strong-consistency model, or re-sharding a Cassandra cluster — these are multi-quarter engineering efforts.

This is why every senior engineering interview returns to databases, why every system design discussion eventually arrives at data storage, and why understanding databases deeply — not just their surface APIs but their internal architectures, tradeoffs, and failure modes — is the clearest differentiator between a mid-level and a senior engineer.

The Four Fundamental Questions

Before you choose a database, before you write a single line of schema or configuration, you must answer four questions in order. These questions are a forcing function that eliminates most wrong choices before you have to think about them.

Question 1: What is the data model?

Is your data relational — rows and columns with foreign-key relationships between entities? Is it hierarchical — nested JSON objects where related data lives inside a parent document? Is it a graph — entities connected by typed, directional relationships? Is it time-ordered — measurements arriving in sequence, each tagged with a timestamp? Is it a high-dimensional vector — the output of a neural network embedding layer?

The shape of your data is the most powerful constraint. A graph database is extraordinarily efficient for traversing six degrees of separation between users, but it is terrible at scanning all users created in the last 30 days. A relational database is superb at joining orders to customers to products in a single query, but modeling a social graph with variable-depth relationships in SQL requires either recursive CTEs that become slow, or a denormalized schema that becomes inconsistent.

Match the data model of your database to the natural shape of your data.

Question 2: What are the access patterns?

How will you read and write this data? Access patterns are often more important than data model. You must enumerate every query your system will run — not abstractly, but concretely:

- "Fetch user by ID" — point lookup by primary key
- "Fetch all orders for user X in the last 30 days, sorted by created_at" — range scan with filter
- "Find all products matching 'wireless headphones' under \$100" — full-text search with a price filter
- "Given user A, find users within 3 degrees of connection" — multi-hop graph traversal
- "Compute daily active users for each country over the last 90 days" — aggregation over large dataset

Each of these access patterns favors a completely different database. A key-value store handles the first perfectly and is completely incapable of the others. An OLAP

warehouse handles the last beautifully but would be absurd for the first. Enumerating access patterns first, then choosing a database that satisfies all of them (or splitting into multiple databases for different patterns), is the correct design process.

Question 3: What are the scale requirements?

"Scale" is not a single number. It is a multidimensional space:

- **Read throughput:** How many reads per second? Is it 100, 100,000, or 10 million?
- **Write throughput:** How many writes per second? Are writes uniform or bursty?
- **Data volume:** How many total records? How much total data in bytes? Is it growing linearly or exponentially?
- **Record size:** Are records small (100 bytes per user session) or large (10 MB per video metadata record)?
- **Read-write ratio:** Is the system 99% reads and 1% writes, or the reverse?
- **Latency SLA:** Does a read need to return in 1ms, 10ms, or 1 second?
- **Geographic distribution:** Do users in both the US and India need low-latency reads and writes?

These numbers dictate architecture. A system with 10,000 reads/sec and 500 GB of data can be served beautifully by a single well-tuned PostgreSQL instance with a Redis cache. A system with 1 million writes/sec across 100 TB of data needs a very different architecture — likely Cassandra or a distributed key-value store.

Never choose a distributed database because it sounds impressive. Single-node databases are faster, simpler, and cheaper for workloads they can handle. Only reach for distributed complexity when the scale genuinely demands it.

Question 4: What consistency guarantee do you need?

This is the question most engineers skip, and it is the one that causes the most production incidents.

- If a user transfers \$10,000 from account A to account B, can any other user ever see a state where the \$10,000 has left account A but not yet arrived in account B? If the answer is no — strong consistency is required.
- If a user posts a tweet and a follower in another region sees it 500 milliseconds later instead of instantly, is that acceptable? If yes — eventual consistency is fine.
- If an inventory counter shows 1 item available and two users simultaneously try to buy it, can both succeed? If the answer is no — you need serializable isolation.

Consistency requirements drive you toward or away from entire categories of databases. They determine whether you can use an eventually-consistent distributed store like Cassandra or need a strongly-consistent system like PostgreSQL or Google Spanner.

Chapter 2: Relational Databases (SQL / RDBMS)

What They Are

Relational databases organize data into tables: two-dimensional structures consisting of rows (records) and columns (attributes). Every table has a schema — a rigid definition of what columns exist, what data types they hold, and what constraints apply (NOT NULL, UNIQUE, FOREIGN KEY). Tables are linked to each other through foreign key relationships, and data across multiple tables is retrieved together using JOIN operations.

The relational model was invented by E.F. Codd at IBM in 1970 and published in his landmark paper "A Relational Model of Data for Large Shared Data Banks." Codd's insight was that data should be organized into normalized relations (tables), and that queries should be expressed declaratively — you describe *what* you want, not *how* to get it — using a language built on relational algebra. That language became SQL (Structured Query Language), standardized in 1986.

Fifty years later, relational databases remain the most widely deployed data storage technology in the world. PostgreSQL, MySQL, Oracle, and SQL Server collectively power the vast majority of production applications. This is not inertia or legacy conservatism — it is because the relational model, with its combination of ACID guarantees, flexible querying, and mature tooling, remains the best fit for the majority of software applications.

Internal Architecture

Understanding how a relational database actually works internally makes you a far more effective system designer, because you understand why certain operations are fast and others are slow.

Storage Engine

Most relational databases use a B-tree-based storage engine. Data is stored on disk in fixed-size pages (typically 8KB in PostgreSQL). Each page holds a set of rows. The B-tree index is a balanced tree structure where leaf nodes contain pointers to the actual row locations on disk (heap pages). A query like `SELECT * FROM users WHERE id = 12345` traverses the B-tree index in $O(\log n)$ time, following pointers until it reaches the leaf node, then does a heap fetch to read the actual row.

PostgreSQL uses a heap storage model: tables are stored as unordered sets of rows (heap files), and all ordering and lookups go through indexes. MySQL's InnoDB storage engine uses a clustered index model: the primary key IS the table storage — rows are physically stored in primary key order. This makes primary key lookups

slightly faster in InnoDB but makes secondary index lookups more expensive (they must resolve through the primary key).

Buffer Pool / Shared Buffer Cache

Databases do not read and write directly to disk for every query. They maintain a large in-memory buffer pool (PostgreSQL calls it `shared_buffers`) that caches frequently accessed pages. A "hot" table — one queried constantly — will have its pages resident in memory, making reads effectively in-memory speeds. Only when you read data not currently in the buffer pool does the database incur actual disk I/O.

This is why the single most impactful tuning parameter for a PostgreSQL instance is `shared_buffers` — typically set to 25% of total RAM. A system with 64 GB of RAM and a 16 GB buffer pool can serve a 15 GB database entirely from memory.

Write-Ahead Log (WAL)

Every write operation — INSERT, UPDATE, DELETE — is first written to the Write-Ahead Log (WAL) before being applied to the actual data pages. The WAL is an append-only sequential file on disk. Sequential writes are dramatically faster than random writes, so WAL writes are fast.

The WAL serves two purposes: crash recovery and replication. On a crash, the database replays the WAL from the last checkpoint to reconstruct any modifications that were in the buffer pool but not yet flushed to disk. For replication, the WAL is streamed to replica servers, which replay it to stay in sync with the primary. This is why PostgreSQL replication is called "streaming replication" — it streams the WAL.

Query Planner and Executor

When you issue a SQL query, the database does not immediately run it. It first parses the SQL into an abstract syntax tree, then passes it to the query planner (also called the optimizer), which generates a query plan — a tree of physical operations (Sequential Scan, Index Scan, Hash Join, Sort, Aggregate) that will be executed to produce the result.

The query planner uses statistics about the data — maintained by the `ANALYZE` command — to estimate how many rows each operation will return, and uses those estimates to choose the cheapest plan. This is why `ANALYZE` matters: stale statistics lead to bad query plans, which lead to slow queries. A query planner that thinks a table has 1,000 rows when it actually has 10 million will choose a plan that is catastrophically wrong.

You can inspect the query plan using `EXPLAIN` and `EXPLAIN ANALYZE`.

Understanding query plans — why the planner chose a sequential scan over an index scan, why it chose a nested loop join over a hash join — is a core skill for database performance.

ACID Properties — The Full Picture

ACID is the defining characteristic of relational databases, but the four properties are often oversimplified. Let's be precise about what each one means.

Atomicity

A transaction is a group of operations that must all succeed or all fail together.

Atomicity guarantees that a transaction is treated as a single, indivisible unit. If any operation in the transaction fails — due to a constraint violation, a network error, or a hardware failure — all operations in the transaction are rolled back to the state before the transaction began.

The classic example: transferring \$1,000 from account A to account B. This involves two writes: decrement account A by \$1,000, increment account B by \$1,000. Without atomicity, a crash between the two writes could leave account A decremented but account B not yet incremented — money vanishes. With atomicity, the transaction either completes both writes or neither.

Atomicity is implemented through the WAL and an undo log (or MVCC, covered below). On rollback, the database uses the undo information to reverse all changes made by the transaction.

Consistency

This is the most misunderstood of the four properties. Consistency in ACID means that a transaction brings the database from one valid state to another valid state, where "valid" is defined by all the integrity constraints you have defined on the schema: NOT NULL constraints, UNIQUE constraints, CHECK constraints, FOREIGN KEY constraints.

Consistency is partly the database's responsibility (enforcing these constraints) and partly the application's responsibility (writing correct transaction logic). If your business rule is "account balance must never go below zero" and you have a CHECK constraint enforcing this, the database will reject any transaction that would violate it, maintaining consistency.

The "C" in ACID is often considered the weakest of the four letters because it depends heavily on the application correctly defining its constraints.

Isolation

Isolation is the property that concurrent transactions execute as if they were serialized — as if they ran one after another rather than simultaneously. Without isolation, concurrent transactions can interfere with each other in ways that produce incorrect results.

The classic concurrency anomalies that isolation prevents (or mitigates, depending on the isolation level):

Dirty Read: Transaction A reads data written by Transaction B, which has not yet committed. If B then rolls back, A has read data that never existed.

Non-repeatable Read: Transaction A reads a row. Transaction B modifies and commits that row. Transaction A reads the same row again and gets a different value. The read was not repeatable.

Phantom Read: Transaction A reads a set of rows matching a condition (e.g., all orders over \$100). Transaction B inserts a new row matching that condition and commits. Transaction A re-reads and now gets a different set of rows — a "phantom" appeared.

Lost Update: Two transactions both read a value, both compute an update based on it, and both write their result. The second write overwrites the first — one update is lost.

SQL defines four isolation levels, in increasing order of strictness:

1. **Read Uncommitted** — No isolation. Dirty reads allowed. Almost never used.
2. **Read Committed** — Default in PostgreSQL and Oracle. Dirty reads prevented; non-repeatable reads allowed.
3. **Repeatable Read** — Default in MySQL InnoDB. Dirty and non-repeatable reads prevented; phantom reads may occur.
4. **Serializable** — Strongest. All three anomalies prevented. Transactions appear to execute serially.

Higher isolation levels cost more in terms of lock contention and throughput. Most applications run at Read Committed or Repeatable Read.

Durability

Once a transaction commits, its changes are permanent — even if the database crashes immediately after. Durability is implemented via the WAL: the transaction's changes are written to the WAL and flushed to disk (fsync'd) before the commit acknowledgment is returned to the client. Even if the server loses power a millisecond after the commit, the changes survive because they're on durable storage and can be replayed from the WAL.

The cost of durability is fsync latency. An fsync forces all dirty pages in the OS write cache to be physically written to the storage device, which takes several milliseconds. This is why write throughput on a single PostgreSQL instance is bounded: you can only fsync so many times per second. This limit drives the need for techniques like write

batching, group commit, and ultimately distributed write paths for very high write throughput.

Multi-Version Concurrency Control (MVCC)

PostgreSQL (and most modern databases) use MVCC instead of traditional read-write locking to implement isolation. MVCC is one of the most important concepts for understanding database performance.

In a traditional locking model, a read of a row acquires a shared lock, which blocks concurrent writes. A write acquires an exclusive lock, which blocks concurrent reads. This maximizes correctness but minimizes concurrency — readers block writers and writers block readers.

MVCC takes a fundamentally different approach: every write creates a new version of the row, rather than modifying the row in place. Every transaction sees a snapshot of the database as it existed at the moment the transaction began. Reads never block writes, and writes never block reads, because readers look at old versions while writers create new ones.

In PostgreSQL, every row has two hidden system columns: `xmin` (the transaction ID that created this version) and `xmax` (the transaction ID that deleted/updated this version, or zero if the row is still current). When a transaction reads a row, PostgreSQL checks whether the row's `xmin` is visible to the transaction's snapshot (was committed before the snapshot was taken) and whether `xmax` is not visible (the row has not been deleted or updated in a way visible to this transaction). This allows each transaction to see a consistent snapshot without taking locks on the data.

The cost of MVCC is dead tuple accumulation. Old row versions cannot be deleted until no active transaction still needs to see them. PostgreSQL's VACUUM process periodically reclaims space from dead tuples. Failing to run VACUUM frequently enough on a write-heavy table leads to table bloat, degraded performance, and in extreme cases (transaction ID wraparound) a database freeze.

When to Use Relational Databases

Relational databases are the right choice in the following situations:

Strong data integrity requirements. Any system where data corruption — even transient, even temporary — is unacceptable. Financial systems, healthcare records, inventory management, order management, user authentication. The ACID guarantees of a relational database make it extremely difficult to accidentally put data into an inconsistent state.

Complex, ad-hoc queries. Systems where the query patterns are varied and not fully known in advance — business intelligence queries, internal admin dashboards, operational reports. SQL's expressiveness (JOINS, window functions, CTEs, subqueries, aggregations) lets you answer questions that were not anticipated when the schema was designed.

Normalized, relational data. When your data is genuinely relational — entities with foreign key relationships, where the same entity appears in multiple contexts — normalization avoids duplication and ensures consistency. A customer record lives in one place; if their email changes, it changes everywhere.

Moderate scale. A well-tuned PostgreSQL instance on modern hardware (64-core, 256 GB RAM, NVMe SSD) can handle 50,000+ reads/sec and 10,000+ writes/sec on real-world workloads. This is sufficient for the vast majority of applications. Do not prematurely distribute.

When Not to Use Relational Databases

Massive write throughput at scale. When writes exceed what a single node can handle, sharding a relational database is painful. Schema changes across shards, cross-shard transactions, and rebalancing are operationally complex. At this point, consider Cassandra or a distributed SQL database.

Flexible/variable schemas. If each entity in your system has significantly different attributes — like an e-commerce product catalog where a shirt has size/color and a

laptop has CPU/RAM/storage — storing this in a relational schema requires either nullable columns (messy), a key-value attribute table (EAV — extremely slow), or JSON columns (giving up relational benefits). A document database is far more natural.

Graph traversal. Modeling and traversing deep graph relationships (friend-of-friend, six degrees of separation, organizational hierarchies) in SQL requires recursive queries that become slow at depth. A graph database is orders of magnitude faster for multi-hop traversals.

Key Databases in This Category

PostgreSQL — The most feature-rich open-source relational database. Supports JSONB for semi-structured data, full-text search, array types, range types, partitioning, logical replication, and extensions (PostGIS for geospatial, pgvector for embeddings, TimescaleDB for time-series). The default choice for new applications.

MySQL (InnoDB) — Historically faster for simple read workloads, MySQL is the most widely deployed web database due to its LAMP stack origins. InnoDB is the only storage engine worth using. MySQL Workbench and ecosystem tooling are mature.

SQLite — A file-based, serverless embedded relational database. The entire database is a single file. Used in mobile apps (iOS and Android both include SQLite), browsers, and applications that need a local database without a server. Not suited for concurrent multi-writer scenarios.

Oracle Database — The enterprise-grade, feature-richest (and most expensive) SQL database. Dominant in banking, insurance, and large enterprise. RAC (Real Application Clusters) provides active-active multi-node operation. Oracle's optimizer is arguably the most sophisticated in the industry.

Microsoft SQL Server — The Windows-ecosystem equivalent of Oracle. Excellent integration with .NET and Azure. T-SQL dialect. Strong analytics with SSAS and SSRS integration.

CockroachDB — Distributed SQL that speaks the PostgreSQL wire protocol. Data is automatically sharded and replicated across nodes using Raft consensus. Provides serializable isolation with horizontal scale. Covered in depth in the NewSQL chapter.

Chapter 3: Document Databases

What They Are

Document databases store data as semi-structured documents — typically JSON or BSON (Binary JSON). Unlike relational databases where data is spread across normalized tables and reassembled via JOINS, a document database encourages you to store related data together in a single document. A user profile document might contain the user's basic info, their preferences, and their recent activity — all in one JSON object.

The defining feature is the flexible schema. Different documents in the same collection (the equivalent of a table) can have completely different fields. Document 1 might have a `phone_number` field; document 2 might not. Document 1 might have a nested `address` object; document 3 might have multiple addresses in an array. This flexibility makes document databases well-suited for domains where entities are diverse, where the schema evolves frequently, or where the data is naturally hierarchical.

The Embedding vs. Referencing Decision

The most important design decision in a document database is how to handle relationships between entities: embedding or referencing.

Embedding means nesting related data directly inside the parent document. Instead of a separate `comments` table with a foreign key to `posts`, you store an array of comment objects directly inside the post document.

```
{
  "_id": "post_123",
  "title": "System Design Tips",
  "author_id": "user_456",
  "content": "...",
  "comments": [
    { "user_id": "user_789", "text": "Great post!", "created_at": "2024-01-15" },
    { "user_id": "user_101", "text": "Very helpful", "created_at": "2024-01-16" }
  ]
}
```

Embedding is ideal when: the embedded data is always accessed with the parent (you always want comments when you fetch a post); the embedded data doesn't need to be queried or updated independently; the number of embedded documents is bounded (a post won't have millions of comments).

Referencing stores the related entity's ID in the document and requires a separate lookup. This is closer to the relational model.

```
{
  "_id": "post_123",
  "title": "System Design Tips",
  "author_id": "user_456",
  "comment_ids": ["comment_001", "comment_002"]
}
```

Referencing is better when: the related data is large or unbounded; the related data needs to be queried independently; the same related entity appears in many parent

documents (a user appears in thousands of posts — you don't want to embed a copy of the user in each post); or you need transactional updates across multiple documents.

The classic MongoDB design rule: **embed when you access data together, reference when you access data separately**. The anti-pattern that causes the most performance problems is using MongoDB like a relational database — storing references everywhere and doing application-level joins.

Internal Architecture — MongoDB as the Reference

MongoDB's storage engine (WiredTiger since 3.2) uses a B-tree for indexed access and a BSON-based document store. Documents are stored in collection files on disk, and indexes are B-trees keyed on specific fields (or sub-fields of nested objects, or elements of arrays).

WiredTiger uses a log-structured merge (LSM) approach internally for writes — a write cache in memory is periodically compacted and merged to disk. This provides excellent write throughput at the cost of read amplification, which WiredTiger mitigates through its block cache.

MongoDB's replication is built on replica sets: a group of MongoDB instances (typically 3 or 5) where one is the primary and the others are secondaries. Writes go to the primary, which replicates to secondaries via an oplog (operations log) in near-real-time. If the primary fails, the secondaries elect a new primary via Raft-like consensus. Read preferences can be configured to read from secondaries (with eventual consistency) for read scaling.

MongoDB's sharding (horizontal scaling) partitions collections across multiple shards using a shard key. The choice of shard key is critical: a shard key with low cardinality or non-uniform distribution causes hotspots. A monotonically increasing shard key (like a timestamp or auto-increment ID) sends all inserts to the same shard — always use a hashed shard key or a compound shard key with good distribution for write-heavy sharded collections.

Multi-Document Transactions

Early MongoDB (before 4.0) had no multi-document transactions — each individual document write was atomic, but you could not update two documents atomically. This was a significant limitation for any workflow that required consistency across entities.

MongoDB 4.0 added multi-document ACID transactions, making the atomicity guarantee similar to a relational database. However, multi-document transactions in MongoDB are significantly more expensive than in PostgreSQL, because they require acquiring document-level locks across potentially multiple shards and coordinating via a two-phase commit protocol. The design philosophy in MongoDB remains: **design your schema to avoid cross-document transactions through good embedding**. Use multi-document transactions only as a last resort.

Firestore — The Serverless Document Database

Google Firestore (formerly Firebase Realtime Database, then Cloud Firestore) is a serverless document database that adds real-time sync capabilities on top of the document model. Documents can be "watched" by clients, and changes propagate to all watching clients in real-time via WebSockets.

Firestore's security model is unique: Firestore Security Rules allow you to define read and write access control directly in the database layer, based on the authenticated user's identity and document fields. This lets mobile and web apps read from and write to Firestore directly, without a backend server — the database enforces security.

Firestore's query model is more limited than MongoDB's: compound queries require composite indexes, and certain query patterns (like aggregations) are not supported natively. But for mobile apps and real-time collaborative applications, Firestore is extremely productive.

When to Use Document Databases

Product catalogs with variable attributes. A catalog with shirts, laptops, furniture, and food items — each category has completely different attributes. A document database lets each product document have exactly the fields relevant to its type, without null columns or EAV tables.

User profiles and preferences. User profile data is naturally hierarchical (profile info → preferences → settings → activity) and varies between users. Document storage is natural.

Content management systems. Articles, blog posts, and pages have varying structure. A post might have a cover image, embedded videos, author bios, tags, categories — all nested naturally in a document.

Mobile app backends with offline sync. Firestore in particular supports offline-first mobile apps where data syncs when connectivity is restored.

Rapid iteration / early-stage products. When the schema is not yet fully known and will evolve rapidly, the schema-flexible nature of document databases avoids painful migrations.

When Not to Use Document Databases

Heavily relational data. If your queries constantly need to join entities from different collections — "all orders with their products with their reviews with their sellers" — you will write application-level join code that is verbose, slow, and error-prone. Use a relational database.

Financial transactions. Multi-document transactions in MongoDB have overhead and complexity. For money movement, use a relational database with proper ACID transactions.

Complex analytics. Aggregation pipelines in MongoDB are powerful but not designed for analytical workloads over hundreds of millions of records. Use an OLAP warehouse.

Chapter 4: Key-Value Stores

What They Are

Key-value stores are the simplest data model in computer science: a distributed dictionary or hash map. You store a value under a key; you retrieve a value by its key. That is the entire data model. There are no tables, no schemas, no relationships, no queries, no joins — just `SET key value` and `GET key`.

The simplicity of this model enables extraordinary performance. With no schema to validate, no query to plan, no indexes to traverse, and no relationships to enforce, key-value stores can process millions of operations per second with sub-millisecond latency — often serving data entirely from RAM.

Redis — A Deep Dive

Redis is not a simple key-value store — it is a data structure server. The value in a Redis key can be one of many rich data types, each with its own set of operations:

String: The basic type. Stores a byte sequence up to 512 MB. Operations: GET, SET, INCR, DECR, APPEND, GETRANGE. Used for: caching serialized objects, counting (atomic INCR), rate limiting.

Hash: A map of field → value within a single key. Like a mini-document stored at one key. Operations: HGET, HSET, HMGET, HGETALL. Used for: user sessions (one hash

per user with fields like `last_seen`, `cart_count`), object representation without serialization.

List: A doubly-linked list of strings. Operations: LPUSH, RPUSH, LPOP, RPOP, LRANGE. Used for: message queues (BLPOP for blocking pop), activity feeds (latest N events), job queues.

Set: An unordered collection of unique strings. Operations: SADD, SMEMBERS, SISMEMBER, SUNION, SINTER, SDIFF. Used for: tracking unique visitors (unique IPs per page), real-time who's online, tagging systems.

Sorted Set (ZSet): A set where each member has an associated float score, and members are ordered by score. Operations: ZADD, ZRANGE, ZRANGEBYSCORE, ZRANK. Used for: leaderboards (score = points, members ordered by rank), rate limiting with timestamps, priority queues, time-series approximations.

Bitmap: Strings treated as bit arrays. Operations: SETBIT, GETBIT, BITCOUNT, BITOP. Used for: daily active user tracking (one bit per user per day, extremely compact — 1 million users = 125 KB), feature flags.

HyperLogLog: A probabilistic data structure for counting unique elements with a fixed memory footprint (~12 KB) regardless of cardinality, with ~0.81% error rate. Used for: approximate unique visitor counting at massive scale.

Stream: An append-only log of entries with auto-generated IDs. Supports consumer groups for distributed message processing. Redis Streams is a lightweight Kafka-like message log.

Redis Persistence

Despite being primarily an in-memory database, Redis supports two persistence mechanisms:

RDB (Redis Database Snapshots): Periodic point-in-time snapshots of the entire dataset to disk. Fast to restore from (a single file), but any writes since the last snapshot

are lost on crash. Configured with: `save 900 1` (save if at least 1 key changed in 900 seconds).

AOF (Append-Only File): Logs every write operation to a file, like a WAL. On restart, Redis replays the AOF to reconstruct the dataset. Three fsync policies: `always` (safest, slowest), `everysec` (lose at most 1 second of data, good balance), `no` (OS decides when to flush, fastest, most data loss risk).

In production, combine both: AOF for durability, RDB for fast restarts.

Redis Replication and Clustering

Redis uses a single-leader replication model: one primary, multiple replicas. Replicas receive and replay the primary's command stream asynchronously. Redis Sentinel provides automatic failover: it monitors the primary and promotes a replica if the primary fails.

Redis Cluster provides horizontal sharding: the keyspace is divided into 16,384 hash slots, and each cluster node owns a range of slots. When you write a key, Redis Cluster computes `CRC16(key) % 16384` to determine which slot (and therefore which node) owns it. Client libraries are slot-aware and route commands to the correct node.

Redis Cluster has important constraints: multi-key operations (MGET, MSET, SUNIONSTORE) only work if all keys hash to the same slot. You can force keys to the same slot using hash tags: `{user:123}:profile` and `{user:123}:session` both use `user:123` as the hash input and land on the same slot.

Rate Limiting with Redis

Rate limiting is one of the most common Redis use cases in system design interviews. Here is the standard pattern:

Fixed window counter: For each user and each time window (e.g., 1-minute windows), maintain a counter. Increment on each request; reject if counter exceeds limit.

```
key = "rate:user:123:2024011514" (user 123, window = 2024/01/15 14:xx)
count = INCR key
EXPIRE key 60
if count > 100: reject
```

Sliding window log: Store a sorted set where the score is the request timestamp. On each request, remove old entries (outside the window), count remaining entries, and add the new one.

```
now = current_timestamp_ms
window_start = now - 60000
ZREMRANGEBYSCORE key 0 window_start
count = ZCARD key
ZADD key now now
EXPIRE key 60
if count >= 100: reject
```

Token bucket (using Lua script for atomicity): Maintain a bucket with a token count and a last-refill timestamp. On each request, compute how many tokens to add since the last refill ($\text{time_elapsed} \times \text{rate}$), cap at `max_tokens`, then check if there's at least one token available.

Lua scripts in Redis execute atomically — no other commands can run between the EVAL start and end — making them essential for multi-step operations like rate limiting that must be race-condition-free.

DynamoDB — Key-Value at AWS Scale

DynamoDB is Amazon's fully managed, serverless key-value and document database. It was designed to handle Amazon's peak retail traffic — millions of requests per second — with single-digit millisecond latency at any scale. Werner Vogels' 2007 "Dynamo" paper describing its architecture became one of the most influential distributed systems papers ever written.

DynamoDB's data model: every item (the equivalent of a document or row) is identified by a primary key, which can be either a partition key alone (hash key) or a composite of a partition key and a sort key.

The partition key determines which physical partition stores the item (DynamoDB uses consistent hashing internally). The sort key allows multiple items with the same partition key to be sorted and range-queried within a partition. A common pattern:

`user_id` as partition key, `created_at` as sort key — lets you retrieve all items for a user, sorted by time, with a single query.

DynamoDB's capacity model: you provision read capacity units (RCUs) and write capacity units (WCUs), or use on-demand pricing that scales automatically. A single WCU handles one 1 KB write per second; a single RCU handles one 4 KB strongly consistent read (or two eventually consistent reads) per second. At massive scale, provisioned capacity with auto-scaling is more cost-effective than on-demand.

DynamoDB Global Tables provides multi-region active-active replication — writes can happen in any region, and data replicates to all other regions asynchronously. This enables globally distributed applications with low-latency local writes. Conflicts (same item written in two regions simultaneously) are resolved by last-writer-wins based on timestamp.

Chapter 5: Wide-Column Databases

What They Are

Wide-column databases (also called column-family stores) organize data into rows, each identified by a row key. But unlike relational databases, columns are not fixed —

different rows can have different columns, and columns are grouped into column families. The schema is sparse: only columns that have values need to be stored.

The mental model: imagine a sorted dictionary where each key maps to another dictionary of column family $\rightarrow$ {column name $\rightarrow$ value}. Or think of a two-dimensional map: `Map<RowKey, Map<ColumnFamily:ColumnName, Value>>`.

The defining design principle of wide-column databases is that the data model must be designed around the queries, not around the entities. In relational databases, you normalize to eliminate redundancy, then query with JOINS. In wide-column databases, you denormalize to match query patterns, accepting redundancy in exchange for query performance.

Apache Cassandra — Architecture in Depth

Cassandra was developed at Facebook to power the inbox search feature, then open-sourced in 2008. It combines the data model from Google's Bigtable with the distributed architecture from Amazon's Dynamo.

Ring Architecture and Consistent Hashing

Cassandra arranges nodes in a logical ring. Each node owns a range of the token space (by default, a 64-bit integer space from -2^{63} to $2^{63}-1$). When data is written, the row key is hashed to a token, and the token determines which node (and its replicas) owns that data.

With virtual nodes (vnodes — enabled by default since Cassandra 1.2), each physical node owns many small token ranges rather than one contiguous range. This provides better load distribution and makes adding/removing nodes less disruptive — only the vnodes of the affected node need to be redistributed.

Write Path

A Cassandra write is one of the fastest write paths of any database:

1. The client contacts any node (the coordinator node for this request).

2. The coordinator determines which nodes are responsible for the data (based on the partition key hash and the replication factor).
3. The write is sent to all replica nodes simultaneously.
4. Each replica node: (a) appends the write to its commit log (sequential disk write — fast); (b) writes to the in-memory memtable; and (c) returns an acknowledgment.
5. The coordinator returns success to the client once the required number of acknowledgments (determined by the write consistency level) are received.

Note: there is no lock, no B-tree write, no single bottleneck. The commit log write is sequential and fast. The memtable is in-memory. This is why Cassandra has extraordinary write throughput.

When a memtable reaches its size threshold, Cassandra flushes it to disk as an SSTable (Sorted String Table). SSTables are immutable — they are never modified after being written. This eliminates write amplification from in-place updates.

Compaction

Over time, a Cassandra node accumulates many SSTables. Reads must check all SSTables for a given partition key (plus the active memtable). Compaction merges multiple SSTables into a new, larger one, removing deleted data (tombstones) and keeping only the most recent version of each cell.

Cassandra supports multiple compaction strategies:

- **SizeTiered**: Default. Merges SSTables of similar size. Good for write-heavy workloads.
- **LeveledCompaction**: Organizes SSTables into levels with size limits. Better read performance (fewer SSTables to check), higher write amplification.
- **TimeWindowCompaction**: Groups SSTables by time window. Ideal for time-series data that expires by age.

Read Path and Bloom Filters

A Cassandra read is more expensive than a write. The node must check the memtable and potentially many SSTables. To avoid reading SSTables that don't contain the

requested partition key, Cassandra maintains a Bloom filter for each SSTable — a probabilistic data structure that tells you (with no false negatives, but possible false positives) whether a key might be in that SSTable.

If the Bloom filter says "no," the SSTable is skipped entirely. This dramatically reduces the number of SSTables read per query. However, Bloom filters do not help with range scans or queries that need to check many partitions.

Consistency Levels

Cassandra's tunable consistency is one of its most powerful features. For each read and write, you specify a consistency level:

- **ONE:** Only one replica must acknowledge.
- **QUORUM:** A majority ($\text{floor}(N/2) + 1$) of replicas must acknowledge.
- **ALL:** Every replica must acknowledge.
- **LOCAL_QUORUM:** A quorum within the local datacenter.
- **EACH_QUORUM:** A quorum in each datacenter.

For strong consistency, use **QUORUM** for both reads and writes when the replication factor is 3: 2 nodes must acknowledge writes, 2 nodes must respond to reads, and since $2 + 2 > 3$, at least one node in both operations is the same — guaranteeing you always read the latest write.

For maximum write availability, use **ONE** for writes — as long as one replica is alive, writes succeed. Combine with **QUORUM** reads to balance.

Data Modeling in Cassandra

Cassandra's query-first data modeling is fundamentally different from relational modeling. You must know your queries before you design your tables.

Rules of Cassandra data modeling:

1. **Design tables to support specific queries.** There is no "I'll figure out queries later." Each query gets its own table, denormalized to make that query efficient.

2. **The partition key determines data distribution.** All data with the same partition key lives on the same set of nodes and is retrieved together. Good partition keys have high cardinality (many distinct values) and are accessed uniformly.
3. **The clustering columns determine row ordering within a partition.** Rows within a partition are sorted by clustering columns. Design clustering columns to match your sort order.
4. **No JOINS, no subqueries, no GROUP BY.** If you need data from multiple tables, the application does the join. If you need data aggregated, pre-aggregate at write time.
5. **Tombstones are expensive.** Cassandra deletions create tombstones — markers saying "this data was deleted." Tombstones accumulate until compaction. Excessive tombstones degrade read performance. Avoid designs that require many deletes.

Example: designing a Cassandra schema for a Twitter-like feed.

Query: "Give me the 20 most recent tweets from users that user X follows, ordered by time."

In Cassandra, you cannot do this with a JOIN. Instead, you maintain a materialized timeline per user: when user A tweets, you write a copy of the tweet to the timeline of every follower of A (fan-out on write). The table:

```
CREATE TABLE user_timeline (  
    user_id UUID,  
    tweet_time TIMESTAMP,  
    tweet_id UUID,  
    content TEXT,  
    author_id UUID,  
    PRIMARY KEY (user_id, tweet_time, tweet_id)  
) WITH CLUSTERING ORDER BY (tweet_time DESC, tweet_id DESC);
```

Query becomes: `SELECT * FROM user_timeline WHERE user_id = ? LIMIT 20` — a single partition read.

The cost: Twitter with 300 million active users, 500 million tweets/day means massive fan-out. A celebrity with 50 million followers causes 50 million writes per tweet. This is why Twitter eventually moved to a hybrid model: materialized timelines for most users, real-time merge of celebrity tweets at read time.

Chapter 6: Graph Databases

What They Are

Graph databases store data as a property graph: a set of nodes (entities) and edges (relationships) between them, where both nodes and edges can have arbitrary properties (key-value pairs).

The critical insight of graph databases is that relationships are first-class citizens. In a relational database, a relationship between two entities is represented by foreign keys — to find related entities, you JOIN tables, which requires scanning index structures. In a graph database, relationships are literally stored as pointers — traversing a relationship means following a pointer in memory, which is $O(1)$ per hop regardless of the total database size.

This is the property of index-free adjacency: a graph database does not look up an index to find neighbors — it follows direct pointers. For queries that traverse many hops (find all friends of friends of friends), this is orders of magnitude faster than SQL JOINS.

Neo4j — Architecture and Cypher

Neo4j stores nodes and relationships as records on disk. Each node record points to its first relationship. Each relationship record is a doubly-linked list that can be traversed

in either direction. This linked-list structure means that traversing all relationships of a node requires following pointers, not reading an index — exactly index-free adjacency.

Cypher is Neo4j's declarative graph query language. Its syntax mirrors the visual representation of a graph, with ASCII art for relationships:

```
(node) - [:RELATIONSHIP_TYPE] -> (other_node)
```

Finding users within 2 degrees of a specific user:

```
MATCH (start:User {id: 'user_123'}) - [:FOLLOWS*1..2] -> (connected:User)
RETURN DISTINCT connected.name, connected.id
```

Finding the shortest path between two users:

```
MATCH path = shortestPath(
  (a:User {id: 'user_123'}) - [:FOLLOWS*] - (b:User {id: 'user_456'})
)
RETURN path
```

Finding fraud rings — circular payment patterns:

```
MATCH (a:Account) - [:SENT_TO] -> (b:Account) - [:SENT_TO] -> (c:Account) - [:SENT_TO] -> (a)
WHERE a.balance < 0
RETURN a, b, c
```

This last query — finding cycles in a graph — would require a recursive CTE in SQL that becomes prohibitively slow for large graphs. In Neo4j, it executes in milliseconds.

Graph Algorithms

Graph databases include built-in implementations of graph algorithms from the Graph Data Science (GDS) library:

PageRank: Measures the importance of nodes based on the importance of nodes that link to them. Originally used by Google for web search ranking. In product recommendation, PageRank identifies influential items. In fraud detection, it identifies high-risk accounts (those connected to many known-fraudulent accounts).

Community Detection (Louvain algorithm): Partitions a graph into communities where nodes within a community have more connections to each other than to nodes outside. Used to find social groups, customer segments, or related product clusters.

Shortest Path (Dijkstra, A*): Finds the minimum-cost path between nodes. Used in routing, delivery optimization, and network topology analysis.

Label Propagation: Spreads labels through the graph based on connectivity. Used to classify unknown nodes based on known neighbors — e.g., if most of an account's transaction partners are flagged for fraud, label propagation will identify the unknown account as likely fraudulent.

Centrality measures: Identify the most "important" nodes. Betweenness centrality (how many shortest paths pass through a node) identifies brokers/bridges. Closeness centrality identifies nodes that can reach others quickly.

When to Use Graph Databases

Fraud detection: Fraud rings exhibit characteristic graph patterns — circular money flows, accounts that share identity attributes, unusually dense connections between accounts. Graph traversal identifies these patterns in real time.

Social networks: Friend-of-friend queries, influence networks, six-degrees-of-separation problems. LinkedIn's "People You May Know" and "Degrees of Connection" features use graph traversal.

Recommendation engines: "Customers who bought X also bought Y" is a bipartite graph problem (customers → products). Graph-based recommendations can incorporate much richer signals than a simple co-occurrence matrix.

Knowledge graphs: Semantic relationships between concepts. Google's Knowledge Graph (the entity information that appears in search results) is a massive property graph. Enterprise knowledge graphs connect products, suppliers, factories, and contracts.

Identity and access management (IAM): "Does user X have permission to access resource Y?" is a graph traversal through roles, groups, and permission hierarchies. A policy tree might be 5–10 hops deep.

Network and IT topology: Modeling infrastructure as a graph — servers, switches, routers, services, dependencies — enables impact analysis ("if this router fails, which services are affected?") via graph traversal.

Chapter 7: Time-Series Databases

What They Are

Time-series databases (TSDBs) are specialized for storing and querying sequences of measurements indexed by time. The defining characteristics of time-series data: observations arrive in approximate time order; each observation is a (timestamp, value, tags) tuple; data is almost exclusively appended, rarely updated, and sometimes deleted in bulk by age (retention policies); and the most common queries are range queries over

time ("CPU usage in the last 30 minutes") and time-based aggregations ("average response time per 5-minute window").

These characteristics allow TSDBs to make architectural choices that general-purpose databases cannot: aggressive compression (because values in a time series are often highly correlated — CPU usage doesn't jump from 50% to 99% between adjacent samples), columnar storage within time blocks, built-in downsampling, and time-range-based data expiration.

The Time-Series Data Model

A time-series is uniquely identified by its measurement name and a set of tags (key-value labels). For example:

```
measurement: cpu_usage
tags: host=server-01, region=us-east, datacenter=dc1
fields: value=73.2
timestamp: 2024-01-15T14:32:00Z
```

Tags identify *what* is being measured (which server, which region). Fields are the measured values. The combination of measurement name and tags defines a unique time series. With 10,000 servers, each reporting 50 metrics, you have 500,000 distinct time series — this is called high cardinality.

High cardinality is the primary scaling challenge for TSDBs. In InfluxDB, each unique combination of tags creates an index entry. With 500,000 time series and 10 million data points per day, the index can grow to billions of entries, consuming enormous memory. TSDBs have cardinality limits and require careful tag design to avoid cardinality explosions. Labels that vary per data point (like a request ID or user ID) should never be tags — use fields instead.

InfluxDB — Architecture

InfluxDB organizes data into time-structured merge trees (TSM). Data is first written to a WAL (for durability) and an in-memory cache. When the cache fills, it's flushed to a TSM file on disk. TSM files are columnar — all values for a field are stored contiguously, enabling both compression and fast aggregation.

InfluxDB's delta-of-delta encoding: timestamps in a series are often equally spaced (one measurement per second). Storing the delta between consecutive timestamps (always 1 second = just 1 value per series) and then the delta of those deltas (always 0 = one value) compresses timestamp storage dramatically. Values use Gorilla compression (from Facebook's TSDB paper) — XOR of consecutive floats, combined with a variable-length encoding of the leading and trailing zeros. Real-world compression ratios: 10:1 to 100:1 for typical metric data.

InfluxDB's retention policies automatically delete data older than a configured threshold. You can have multiple retention policies: keep raw data for 7 days, keep hourly averages for 90 days, keep daily averages forever. Continuous queries automatically compute aggregates and downsampled data at defined intervals.

Prometheus — The Pull Model

Prometheus is the standard monitoring system for cloud-native applications. Unlike most databases (which receive data pushed to them), Prometheus uses a pull model: Prometheus scrapes HTTP endpoints (metrics endpoints) on each monitored service at a configured interval (typically 15–60 seconds).

The pull model has important advantages: Prometheus knows immediately if a target is unreachable (the scrape fails); there is no need to configure each service with the Prometheus server's address; and the scrape interval is controlled centrally. The trade-off: services must expose an HTTP endpoint and be reachable by Prometheus.

PromQL (Prometheus Query Language) is a functional query language designed for time-series data:

- `http_requests_total{job="api-server", status="500"}` — select a specific metric with labels
- `rate(http_requests_total[5m])` — per-second rate over 5 minutes
- `histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))` — 99th percentile latency
- `sum by (datacenter)(rate(http_requests_total[5m]))` — aggregate across instances by datacenter

Prometheus's local storage is a custom TSDB (implemented by Fabian Reinartz, also the author of Thanos). Data is organized into 2-hour time blocks, each an independent mini-TSDB. Long-term storage is handled by remote_write to a separate system (Thanos, Cortex, Mimir) — Prometheus itself is not designed for multi-year data retention.

TimescaleDB — SQL for Time-Series

TimescaleDB is a PostgreSQL extension that adds time-series capabilities to a standard relational database. Instead of learning a new query language or tooling, you use standard SQL — but with automatic data partitioning (chunking), time-series-specific functions, and continuous aggregates.

TimescaleDB automatically partitions tables into time-based chunks. A query like `SELECT avg(cpu) FROM metrics WHERE time > NOW() - INTERVAL '1 day'` touches only the last-day chunk, not the entire multi-year table. This is the key performance insight: time queries have natural locality — recent data is in recent chunks.

Continuous aggregates in TimescaleDB are pre-computed, automatically refreshed materialized views. Define a view that computes hourly averages, and TimescaleDB

incrementally updates it as new data arrives — you query the aggregate instead of raw data, with instant response times even over years of history.

TimescaleDB is the best choice when: you want SQL compatibility; you have mixed workloads (both time-series and relational data); you want to join metrics with your application data; or your team is already familiar with PostgreSQL.

Chapter 8: Search and Full-Text Engines

What They Are

Search engines are built on inverted indexes. In a normal (forward) index, you look up a document ID to find its contents. In an inverted index, you look up a term to find all document IDs that contain that term. This inversion is what makes full-text search fast: to find all documents containing "database" AND "performance," you look up both terms in the inverted index (two index lookups) and intersect the resulting document ID lists.

Beyond basic full-text search, modern search engines provide relevance ranking (not just "does this document contain the term" but "how relevant is this document to the query"), faceted filtering (filter by price, category, brand simultaneously), autocomplete and typeahead, fuzzy matching (finding "databaze" even if the user misspelled "database"), and increasingly, vector/semantic search.

Elasticsearch — Architecture Deep Dive

Elasticsearch is built on top of Apache Lucene, a Java library that provides the core inverted index, relevance scoring, and query execution. Elasticsearch adds a distributed layer, a REST API, and horizontal scaling on top of Lucene.

Indexing Pipeline

When a document is indexed, it goes through an analysis pipeline:

1. **Character filter:** Pre-processing on raw text (e.g., strip HTML tags, convert "&" to "and").
2. **Tokenizer:** Split text into tokens (individual words, or n-grams). The standard tokenizer splits on whitespace and punctuation.
3. **Token filters:** Transform tokens: lowercase, remove stop words ("the", "a", "is"), stemming (reducing "running" → "run"), synonym expansion.

The resulting tokens are stored in the inverted index with their positions (which position in the document each term appears — needed for phrase queries like "system design").

BM25 Relevance Scoring

Elasticsearch uses BM25 (Best Match 25) for relevance scoring, an improvement over classic TF-IDF. BM25 scores a document for a query by:

- **Term frequency (TF) saturation:** The more times a term appears in a document, the more relevant — but with diminishing returns. In BM25, the TF component saturates: the 100th occurrence adds much less relevance than the 1st.
- **Inverse document frequency (IDF):** Terms that appear in many documents are less discriminating (the word "the" appears everywhere and is a poor signal). Terms that appear in few documents are strong signals.
- **Field length normalization:** A term in a short document is more significant than the same term in a long document.

Distributed Architecture

An Elasticsearch cluster consists of nodes, each running one or more shards. An index is divided into primary shards (horizontal partitions of the data), and each primary shard has zero or more replica shards (copies, on different nodes, for fault tolerance and read scaling).

When you search, the query is broadcast to one copy (primary or replica) of every shard. Each shard executes the query locally and returns a ranked list of top-N document IDs and scores. The coordinating node merges these partial result sets, re-ranks them globally, and fetches the actual documents for the final top-N results (the "fetch phase").

This architecture means that search latency is bounded by the slowest shard (the "straggler" problem). With many shards, the probability of at least one slow shard increases. Over-sharding (too many shards) is a common Elasticsearch antipattern: Elasticsearch recommends a shard size of 10–50 GB.

Aggregations

Elasticsearch's aggregations framework is one of its most powerful features:

Terms aggregation: Count documents per unique value of a field — like a GROUP BY. "How many products in each category?"

Histogram aggregation: Bucket documents by numeric range. "Distribution of product prices in \$10 buckets."

Date histogram: Bucket by time intervals. "Number of orders per day in the last month."

Nested aggregations: Aggregate within the results of another aggregation. "Average price per category." (Terms aggregation with an avg metric aggregation nested inside.)

Aggregations execute on the inverted index in a single pass and are extremely fast — hundreds of millions of documents can be aggregated in seconds.

The ELK Stack

The most common production Elasticsearch deployment is the ELK stack:

- Elasticsearch — storage and search
- Logstash — data ingestion pipeline (now often replaced by Filebeat + Kafka)
- Kibana — visualization and dashboards

Logs flow from applications → Filebeat (lightweight log shipper) → Kafka (buffer for backpressure) → Logstash (parse and transform) → Elasticsearch. Kibana provides Discover (log exploration), Visualize (charts and dashboards), and Dashboard.

A well-designed ELK stack can ingest millions of log lines per second. Key operational considerations: manage index lifecycle (hot → warm → cold → delete) using ILM (Index Lifecycle Management); use index templates to apply consistent mappings and settings; monitor JVM heap usage (Elasticsearch is Java — heap pressure causes GC pauses that block operations).

Chapter 9: Analytical Databases (OLAP / Data Warehouses)

OLTP vs OLAP — The Fundamental Split

Every large system eventually splits its data infrastructure into two planes: the operational plane (OLTP — Online Transaction Processing) and the analytical plane (OLAP — Online Analytical Processing).

OLTP systems serve the real-time, user-facing application. They are optimized for point lookups and small, frequent transactions. Read a single user's profile; write one

order; update a product's inventory. The working set is often a small fraction of total data. Queries are short, fast (milliseconds), and high-concurrency.

OLAP systems serve analytical queries — business intelligence, reporting, data science. Read 100 million orders to compute daily revenue by product category and region. These queries touch enormous amounts of data but return small results. They run less frequently (minutes to hours) and can tolerate higher latency. They would devastate an OLTP database — a single analytics query that full-scans 100 million rows would lock out all other queries.

This is why you must separate them. The standard architecture: OLTP database serves the application; a nightly (or real-time) ETL/ELT pipeline copies data to the OLAP warehouse; analysts and dashboards query the warehouse, never the application database.

Columnar Storage — The Core Innovation

Row-oriented storage (used by OLTP databases) stores all fields of a row contiguously on disk. Reading a row is one seek + one sequential read. Reading 10 columns of 1 million rows requires reading 1 million row records.

Column-oriented storage stores each column contiguously. Reading the **revenue** column for 1 million orders means reading one contiguous block on disk, skipping all other columns entirely. If you only need 5 of 50 columns (the analytics query never needs all columns), you read 10% of the data.

Column storage also achieves dramatically better compression:

- All values in a column are the same data type (better compression algorithms can be applied)
- Values in a column are often correlated (all **country** values for US orders are "US") — run-length encoding reduces these to a single value plus a count
- Sorted columns can be delta-encoded (store the difference between consecutive values rather than absolute values)

- Dictionary encoding: replace all distinct string values with small integer codes (store a dictionary mapping code → string separately)

ClickHouse and Redshift achieve 10:1 to 100:1 compression ratios on real data, compared to 2:1 to 5:1 for row-oriented storage.

Snowflake — Modern Cloud Data Warehouse

Snowflake is a multi-tenant cloud data warehouse that separates storage from compute. Storage is in Amazon S3 (or Azure Blob / GCS), organized as columnar-compressed micro-partitions (50–500 MB each, automatically clustered). Compute is provided by "virtual warehouses" — clusters of EC2 instances that read micro-partitions from S3, execute queries in memory, and write results back.

The key innovation: multiple virtual warehouses can simultaneously query the same data in S3 without contention. The data engineering team running nightly batch ETL and the BI team running ad-hoc dashboard queries do not share compute resources and do not slow each other down. Each team gets their own virtual warehouse, sized to their workload.

Snowflake's result caching: if the same query is executed twice and the underlying data hasn't changed, Snowflake returns the cached result from the first execution instantly — no compute charge. This makes repeated dashboard refreshes essentially free.

Zero-copy cloning: Snowflake can "clone" a database, schema, or table in seconds — the clone shares the underlying micro-partitions with the original until someone writes to the clone. This makes development, testing, and data science experimentation safe and instant: clone production, experiment freely, delete the clone when done, pay only for changed data.

ClickHouse — Real-Time OLAP

ClickHouse, developed at Yandex, takes a different approach than Snowflake: it is designed for real-time analytics — queries over data that was inserted seconds or minutes ago, not hours or days ago.

ClickHouse's MergeTree engine family stores data in columnar, sorted, compressed parts on disk. New data is written as new parts (fast, no update needed); ClickHouse periodically merges small parts into larger ones in the background (like Cassandra's compaction). Queries read from all current parts and merge results.

The primary key in MergeTree (the ORDER BY clause) sorts data within parts, enabling sparse primary key indexing. ClickHouse does not read the primary key for every row — it stores one index entry per ~8,000 rows (granule). A query filtering on the primary key reads only the relevant granules — dramatically reducing I/O.

ClickHouse's vectorized query execution processes data in batches of 8,000–16,000 values at a time, using SIMD (Single Instruction, Multiple Data) CPU instructions to operate on multiple values simultaneously. This achieves near-theoretical memory bandwidth utilization — ClickHouse can aggregate 1 billion rows per second on a single server.

When to use ClickHouse vs Snowflake: ClickHouse for real-time analytics (seconds of ingestion-to-query latency), sub-second query response on large datasets, and self-hosted deployments. Snowflake for multi-team analytics platforms, complex ETL pipelines, easy managed operations, and compliance requirements.

DuckDB — Analytical Queries in Process

DuckDB is a recent (2019) embedded analytical database. Like SQLite for OLTP, DuckDB requires no server — it is a library embedded in your application. But unlike SQLite (row-oriented, OLTP-focused), DuckDB is columnar and designed for analytical queries.

DuckDB can query CSV, Parquet, and JSON files directly without loading them into a database. This makes it extraordinarily productive for data science: instead of loading a 10 GB CSV into a database, you run SQL directly on the file. DuckDB achieves better performance on many analytical benchmarks than full-featured database servers, because it has zero network overhead and perfectly leverages the local machine's memory and CPU.

DuckDB's integration with Python and R makes it the default embedded analytics engine for data science workflows, replacing pandas for complex aggregations (DuckDB is orders of magnitude faster than pandas on large datasets).

Chapter 10: Vector / Embedding Databases

What They Are

Vector databases store and search high-dimensional vectors — the numerical representations of text, images, audio, and other data produced by machine learning models. These dense vectors (typically 768 to 4096 dimensions) encode semantic meaning: similar concepts have vectors that are geometrically close in the high-dimensional space.

The core query in a vector database is not "find rows where field = value" but "find the K vectors most similar to this query vector." This is the K-Nearest Neighbor (KNN) problem — and at scale (millions of vectors), exact KNN is too slow. Vector databases use Approximate Nearest Neighbor (ANN) algorithms that trade a small amount of recall (occasionally missing the true nearest neighbor) for dramatic speedup.

Why This Matters Now — LLMs and RAG

The explosion of large language model (LLM) applications has made vector databases central to the AI stack. The key pattern is Retrieval-Augmented Generation (RAG):

1. At indexing time: take your knowledge base (documents, FAQs, product descriptions, code), split each into chunks, embed each chunk using an embedding model (e.g., OpenAI `text-embedding-3-large` , producing a 3072-dimensional vector), and store the vectors in a vector database.
2. At query time: when a user asks a question, embed the question using the same model, search the vector database for the K most similar chunks (semantic search), retrieve those chunks as context, and pass them along with the user question to an LLM (GPT-4, Claude, Gemini) to generate a grounded, factual answer.

RAG solves the two biggest problems with raw LLMs: hallucination (the LLM invents facts not in its training data) and knowledge cutoff (LLMs don't know about events after their training date). By providing relevant retrieved context, you give the LLM the facts it needs to answer accurately.

ANN Algorithms

HNSW (Hierarchical Navigable Small World)

HNSW is the dominant ANN algorithm used by most vector databases. It builds a multi-layer graph where:

- Layer 0 contains all vectors with connections to nearby neighbors.
- Higher layers contain progressively fewer vectors, with longer-range connections.

Search starts at the top layer with a small number of entry points. At each layer, the algorithm greedily traverses the graph toward the query vector, then descends to the next layer. The multi-layer structure allows the algorithm to quickly navigate to the right neighborhood (high layers) and then precisely find the nearest neighbors (layer 0).

HNSW achieves excellent recall (95–99% of true nearest neighbors) with high throughput (millions of queries per second on modern hardware). The cost is memory:

the entire HNSW graph must reside in RAM, requiring ~100 bytes per vector per stored vector for the graph structure, plus the vector storage itself (4 bytes × 1536 dimensions × 1M vectors = ~6 GB for the vectors alone).

IVF (Inverted File Index)

IVF divides the vector space into clusters using k-means clustering. Each vector is assigned to its nearest cluster centroid. At query time, the system identifies the `nprobe` nearest centroids to the query vector and searches only the vectors in those clusters.

IVF is more memory-efficient than HNSW (clusters can be stored on disk, only searching relevant clusters) but has lower recall at the same speed. The `nprobe` parameter trades off speed vs recall: higher `nprobe` = higher recall but slower queries.

In practice, most production vector databases use HNSW for online serving (low latency, high recall) and IVF or disk-based variants for large datasets that don't fit in RAM.

Hybrid Search

Pure vector similarity search has a weakness: it finds semantically similar content but can miss exact keyword matches. If a user searches for a specific product code ("iPhone 15 Pro SKU-X7892"), semantic search might return other iPhone models that are semantically similar but not the exact match.

Hybrid search combines vector similarity with keyword (BM25) search, then merges the results using reciprocal rank fusion (RRF) or a learned reranker. The combination produces better results than either alone for most real-world queries.

Weaviate and Qdrant both support hybrid search natively. For pgvector users, you can implement hybrid search by querying with both the `<=>` vector operator and `to_tsquery` for full-text, then ranking with RRF in SQL.

pgvector — Vector Search in PostgreSQL

pgvector is a PostgreSQL extension that adds a vector data type and two index types: IVFFlat and HNSW. For small-to-medium vector stores (up to ~1 million vectors), pgvector is often the pragmatic choice: you get vector search in the same PostgreSQL instance as your application data, with standard SQL, no additional infrastructure, and ACID transactions.

```
CREATE TABLE documents (  
  id BIGSERIAL PRIMARY KEY,  
  content TEXT,  
  embedding VECTOR(1536) -- OpenAI ada-002 embedding dimension  
);  
  
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops);  
  
SELECT id, content, 1 - (embedding <=> '[0.1, 0.2, ...]':vector) AS similarity  
FROM documents  
ORDER BY embedding <=> '[0.1, 0.2, ...]':vector  
LIMIT 10;
```

For larger vector stores or when you need advanced filtering and metadata management, dedicated vector databases like Pinecone, Qdrant, or Weaviate provide better performance and operational tooling.

Chapter 11: NewSQL / Distributed SQL

The Problem They Solve

For decades, engineers faced a binary choice: a relational database with ACID guarantees but limited horizontal scalability, or a NoSQL database with horizontal scale but no ACID and a worse query model. NewSQL databases attempt to break this dichotomy by providing the SQL interface and ACID guarantees of a relational database with the horizontal scalability of NoSQL.

The key architectural insight: the reason traditional relational databases don't scale horizontally is not fundamental to the relational model — it is an artifact of their single-node, shared-memory architecture. If you redesign the storage engine and replication mechanism from scratch to be distributed, you can provide SQL and ACID across many nodes.

CockroachDB — Distributed SQL Over Raft

CockroachDB's architecture: the entire key-value space is divided into ranges (64 MB by default). Each range is replicated to three or more nodes via the Raft consensus protocol. Any node can receive any query — it routes it to the appropriate range leader(s). For queries that span multiple ranges, CockroachDB uses a distributed transaction protocol.

The Raft consensus protocol ensures that every write is confirmed by a majority of replicas before being committed. This is what provides strong consistency: you cannot read a stale value, because any read goes through the range leader, which has the latest committed state.

CockroachDB uses MVCC (similar to PostgreSQL) for concurrent access, with hybrid logical clocks (HLC) to provide consistent timestamps across nodes without a

centralized clock. Serializable isolation is the default — the strongest isolation level, providing no anomalies, even across distributed transactions.

The cost: latency. A distributed transaction that involves multiple ranges — updating two accounts in a money transfer, for example — requires at least two round trips across the network (prepare + commit), plus the Raft consensus round trip within each range's replica group. This means a distributed transaction in CockroachDB takes tens of milliseconds even on a LAN, versus hundreds of microseconds for the same transaction in PostgreSQL.

Google Spanner — Globally Distributed SQL

Google Spanner is the database underlying Gmail, Google Photos, Google Cloud SQL, and much of Google's infrastructure. It provides externally consistent (serializable + real-time ordering) distributed transactions across a global deployment.

Spanner's most famous innovation is TrueTime: a globally synchronized clock provided by GPS receivers and atomic clocks in every Google datacenter. TrueTime exposes not a single timestamp but an interval [earliest, latest] representing the range within which the true current time definitely falls. The uncertainty bound is typically less than 7 milliseconds.

By using TrueTime, Spanner can assign globally consistent commit timestamps: it waits out the clock uncertainty (up to 7ms) before committing, ensuring that the commit timestamp is unambiguously in the past. This wait is what makes Spanner's linearizability externally consistent — causality is preserved globally without requiring a central clock server. The "commit wait" is the price of global consistency.

Spanner's throughput and latency are extraordinary at Google's scale, but the managed version (Cloud Spanner) is expensive relative to alternatives. For most applications, CockroachDB or YugabyteDB provides similar guarantees at lower cost.

Chapter 12: Object and Blob Storage

What It Is

Object storage is a flat namespace for storing binary objects — files, images, videos, documents, backups, model artifacts, log archives. Unlike a filesystem (hierarchical directory tree) or a database (structured, queryable), object storage organizes data as objects in named "buckets." Each object has a key (path-like name), a value (the raw bytes), and metadata (content type, size, custom attributes).

Object storage is not a database in the traditional sense: you cannot query the contents of objects, you cannot filter by field values, you cannot join objects. You get and put objects by key. But for the right use cases, object storage is the only rational choice: it is infinitely horizontally scalable, extremely cheap per GB (orders of magnitude cheaper than block storage or databases), highly durable (S3 promises 11 nines of durability — losing data is virtually impossible), and globally accessible via HTTP.

S3 — Internal Architecture

Amazon S3 stores objects in a distributed, fault-tolerant system across multiple Availability Zones within a region. S3's core guarantees: every object is durably stored by synchronously replicating to at least three AZs before returning a success response. A single AZ failure does not cause data loss or availability impact.

S3 uses consistent hashing to distribute objects across storage nodes. Object keys are hashed to determine placement. This is why the old advice to "prefix keys with random characters" existed: sequential keys (like timestamps) would hash to the same range of nodes, creating hotspots. AWS has since improved load balancing internally, but high-

throughput workloads (thousands of requests per second to the same prefix) can still benefit from random prefixes.

As of December 2020, S3 provides strong read-after-write consistency: immediately after a successful PUT, any subsequent GET of the same key returns the new object. This replaced the previous eventual consistency model for new objects and eliminates a class of bugs where applications would read stale data after writing.

Pre-Signed URLs — The Correct Upload Pattern

A common system design mistake: having the client upload a file to your application server, which then uploads it to S3. This requires the file to traverse your application server twice (download to server + upload to S3), wasting bandwidth, adding latency, and creating scaling bottlenecks on your server.

The correct pattern: generate a pre-signed URL.

1. Client requests an upload URL from your API server.
2. Your API server calls S3 to generate a time-limited pre-signed PUT URL for a specific key.
3. The URL is returned to the client.
4. The client uploads directly to S3 using the pre-signed URL — bypassing your server entirely.
5. S3 notifies your server (via S3 Event Notifications → SQS/Lambda) when the upload completes.
6. Your server verifies the upload and records the S3 key in your database.

This scales infinitely: your server handles only tiny API calls, S3 handles all the bandwidth-intensive uploads directly from clients.

Chapter 13: CAP Theorem, PACELC, and Distributed Systems Theory

CAP Theorem — The Precise Definition

The CAP theorem, stated by Eric Brewer in 2000 and formally proved by Gilbert and Lynch in 2002, says: a distributed data store can provide at most two of the following three guarantees simultaneously:

Consistency (C): Every read receives the most recent write or an error. This is the "linearizability" or "strong consistency" definition — not the "C" in ACID. In CAP, consistency means that the distributed system behaves as if it were a single node with a single copy of the data.

Availability (A): Every request receives a (non-error) response, but without the guarantee that it contains the most recent write. Every non-failed node must return a response to every request.

Partition Tolerance (P): The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

The critical insight: **partition tolerance is not optional**. In any distributed system deployed over a real network (including a datacenter network, not just the internet), network partitions will occur. Cables are cut, switches fail, firewalls misbehave, software bugs cause nodes to stop responding. A system that cannot tolerate partitions will be unavailable whenever a partition occurs — which violates availability anyway.

Therefore, the real choice is: when a network partition occurs, do you prefer to maintain **consistency** (refuse to answer queries for the partitioned portion of the system) or **availability** (answer queries with potentially stale data)?

CP Systems

In a CP system, when a partition occurs, nodes on the minority side of the partition either refuse requests or return errors. They cannot service requests without risking inconsistency. The majority side continues normally.

Examples:

- **HBase, Zookeeper, etcd:** These use leader election (Paxos/Raft). During a partition, the minority side loses the leader and cannot elect a new one (since it doesn't have a quorum), so it rejects all writes and reads.
- **PostgreSQL:** A primary with replicas will continue serving the primary side. Replicas that cannot reach the primary will fall back to read-only mode (if configured) or be unavailable.

AP Systems

In an AP system, when a partition occurs, all nodes continue to respond to requests — but nodes on different sides of the partition may diverge, serving different data. When the partition heals, the system must reconcile the diverged state through conflict resolution.

Examples:

- **Cassandra:** Nodes on both sides of a partition accept reads and writes. When the partition heals, the anti-entropy process (using Merkle trees) identifies and reconciles differences. Conflicts resolved by last-write-wins (LWW) using timestamps.
- **CouchDB:** Uses a multi-version approach with explicit conflict detection; the application must resolve conflicts.
- **DynamoDB:** Eventually consistent by default; strongly consistent reads available at higher latency and cost.

PACELC — A More Complete Model

The problem with CAP is that it only applies during partitions. But partitions are rare in a well-operated datacenter. What about the tradeoffs in normal operation (no partition)?

Daniel Abadi proposed PACELC (2012): **P**artition → **A**vailability vs **C**onsistency; **E**lse (no partition) → **L**atency vs **C**onsistency.

In normal operation, providing strong consistency requires coordination between replicas — a write must be confirmed by multiple replicas before returning success. This coordination takes time (network round trips), adding latency. Providing low latency means returning before all replicas have confirmed, allowing stale reads.

Examples:

- **DynamoDB (default):** PA/EL — During partition: available (AP). Without partition: low latency (EL). Read a consistent snapshot but potentially stale.

- **Spanner:** PC/EC — During partition: consistent (CP, refuse during partition). Without partition: consistent (EC). Waits for quorum confirmation; commit wait for global consistency.

- **Cassandra (QUORUM):** PC/EC — Tuned to consistency: quorum writes + quorum reads.

- **Cassandra (ONE/ONE):** PA/EL — Maximum availability and minimum latency, maximum staleness.

PACELC provides a more honest characterization of database behavior, because it captures the latency-consistency tradeoff that governs normal-operation performance, not just the exotic partition scenario.

Consistency Levels — The Spectrum

Consistency is not binary (consistent vs inconsistent) — it is a spectrum:

Linearizability (strongest): Every operation appears to take effect atomically at some point between its invocation and response. The system appears as a single, atomic register. Google Spanner provides this globally.

Sequential consistency: All operations appear to have executed in some sequential order, and the order is consistent across all clients. Weaker than linearizability — allows reordering as long as the order is the same for all clients.

Causal consistency: Operations that are causally related are seen in order by all clients. Causally unrelated operations may be seen in different orders by different clients. Amazon's COPS system provides causal consistency at geo-distributed scale.

Eventual consistency (weakest): If no new updates are made to a given piece of data, eventually all replicas will return the same value. There are no guarantees about how long "eventually" takes or what intermediate values clients will see.

Eventual consistency enables the highest availability and lowest latency but requires the application to handle stale reads gracefully. It is appropriate for: social media like counts, product view counters, recommendation scores — cases where a slightly stale value is acceptable.

Chapter 14: ACID vs BASE — Consistency Models in Depth

ACID Revisited — Isolation Levels and Anomalies

The ACID model was covered in Chapter 2, but isolation levels deserve much deeper treatment.

Serializable Isolation — What It Actually Means

Serializable isolation guarantees that the result of executing concurrent transactions is equivalent to some serial (sequential) execution of those transactions. It does not mean transactions actually run serially — it means that any observable outcome is indistinguishable from some serial execution.

PostgreSQL's implementation: Serializable Snapshot Isolation (SSI). SSI extends MVCC with a conflict tracking mechanism. Each serializable transaction reads from its snapshot (like repeatable read), but SSI additionally tracks which transactions have read which data that might have been written by other concurrent transactions (read-write conflicts). If SSI detects that the execution of a set of concurrent transactions cannot be explained by any serial ordering, it aborts one of the conflicting transactions with a "serialization failure" error. The application must then retry the transaction.

SSI is optimistic: it allows transactions to proceed concurrently and only aborts on actual conflicts. For most workloads where conflicts are rare, SSI provides serializable isolation with much better throughput than traditional two-phase locking (2PL), which acquires locks before reading and holds them until commit.

The Lost Update Problem — And How to Prevent It

One of the most common concurrency bugs, even at Read Committed isolation. Two transactions read a value, compute an update, and write back — the second write overwrites the first:

```
T1: balance = SELECT balance FROM accounts WHERE id = 1 -- reads 100
T2: balance = SELECT balance FROM accounts WHERE id = 1 -- reads 100
T1: UPDATE accounts SET balance = 100 + 50 WHERE id = 1 -- writes 150
T2: UPDATE accounts SET balance = 100 + 30 WHERE id = 1 -- writes 130 (overwrites)
```

Account should be 180, but ends up at 130.

Solutions:

1. **Atomic operations:** `UPDATE accounts SET balance = balance + 50 WHERE id = 1` — the database reads and writes atomically, no application-level read-modify-write.
2. **SELECT FOR UPDATE:** Acquires an exclusive lock on the selected rows, preventing other transactions from modifying them until the current transaction commits.
3. **Optimistic locking:** Add a `version` column. `SELECT` the version; `UPDATE` with `WHERE version = {selected_version}`. Check that exactly one row was updated

(if zero, another transaction modified it — retry).

4. **Serializable isolation:** SSI will detect and abort conflicting transactions.

Write Skew — The Subtle Anomaly

Write skew is a concurrency anomaly that occurs at Repeatable Read isolation level but not at Serializable. Two transactions read overlapping data, each decides to write based on what it read, and the writes are to different objects — so they don't conflict, but the combination of both writes violates an invariant.

Classic example: hospital on-call rule. At least one doctor must be on call at all times. Both doctors are currently on call. Each checks: "are there at least two doctors on call? If yes, I can take myself off call." Both see two doctors on call. Both take themselves off call. Now zero doctors are on call — invariant violated.

Write skew cannot be prevented with SELECT FOR UPDATE (they wrote to different rows). It requires either serializable isolation or an application-level check that locks the row representing "number of on-call doctors."

BASE — What It Means in Practice

BASE is not a formal model like ACID — it is more of a design philosophy for systems that prioritize availability and performance over strong consistency. The properties are:

Basically Available: The system appears to work most of the time. Failures are partial — some nodes may be down, some data may be stale, but the overall system remains accessible.

Soft State: The state of the system may change over time, even without new input. Replicas converge toward consistency asynchronously. The state is "soft" — it may differ between replicas and change as they synchronize.

Eventually Consistent: Given enough time without new writes, all replicas will converge to the same value. There is no guarantee on how long this takes.

In practice, building on eventually consistent systems requires defensive programming:

Idempotent operations: Make every write idempotent — applying it twice produces the same result as applying it once. If a network retry causes a write to execute twice, idempotency prevents incorrect double-effects.

Commutative operations: Operations that can be applied in any order with the same result. Incrementing a counter is commutative (regardless of order, 5 increments to 20 results in 25). Overwriting a value is not (the order determines the result). Design updates to be commutative where possible.

Conflict detection and resolution: When two writes to the same data diverge, the system must detect the conflict (using vector clocks, Lamport timestamps, or version vectors) and resolve it. Common resolution strategies: last-write-wins (simple, but can lose data), merge functions (CRDT-style), or surface the conflict to the user.

Read-your-writes consistency: After you write data, you expect to read your own write. In an eventually consistent system, this is not guaranteed unless you route reads to the same replica you wrote to, or use session tokens to track which version of data you need.

Chapter 15: Replication — Internals and Patterns

Single-Leader Replication

The most common replication pattern. One node (the leader/primary/master) accepts all writes. The leader replicates changes to follower nodes (replicas/secondaries/slaves). Reads can be served by followers.

Synchronous vs Asynchronous Replication

Asynchronous replication (most common): The leader writes to its WAL, applies the change, and returns success to the client before the replicas have confirmed receipt of the change. Replicas lag behind the leader — "replication lag." The lag is typically milliseconds but can grow to seconds or minutes under high write load or slow network.

Consequence: if the leader fails and a follower is promoted, any writes that were not yet replicated are lost. This is the fundamental durability tradeoff of async replication.

Synchronous replication: The leader waits for at least one replica to confirm before returning success. The client's write is durable on at least two nodes. If the leader fails and the synchronous replica is promoted, no data is lost.

Consequence: write latency is now bounded by the network round trip to the replica. If the replica is slow or the network degrades, writes slow down. If the synchronous replica fails, the leader cannot complete writes until the replica recovers or you switch to async.

Semi-synchronous (PostgreSQL's `synchronous_commit`): Configurable per transaction. You can mark critical transactions (financial writes) as synchronous and routine transactions as asynchronous — best of both worlds at the application level.

Replication Lag and Read-After-Write Consistency

Replication lag causes the following problem: user writes data, then reads it from a replica, and the read returns stale data (the replica hasn't received the write yet). The user refreshes their post and it's gone — terrifying.

Solutions:

Read-your-own-writes (monotonic reads): After a write, route reads for that user to the leader (or to the replica that confirmed the write) for a short window. In practice: store a "last_write_timestamp" in the user's session. If the replica's replication lag is greater than (now - last_write_timestamp), route to leader instead.

Monotonic reads: Ensure a user always reads data at least as recent as what they've read before. Implement by always routing a specific user's reads to the same replica (consistent routing by user ID).

Read from leader only: Highest consistency, no replication lag impact. Cost: leader must serve all reads, losing the read-scaling benefit of replicas.

Multi-Leader Replication

Multiple nodes accept writes. Changes replicate to all other leaders. Used for:

Multi-datacenter deployments: One leader per datacenter. Writes go to the local leader (low latency). Leaders replicate to each other across the WAN. Reads from local replica (low latency). Writes succeed even if the WAN link between datacenters is disrupted.

Offline-capable clients: Mobile apps that can modify data while offline and sync when reconnected. Each device is effectively a leader. CouchDB supports this model.

The fundamental problem: **write conflicts**. If two leaders simultaneously accept different writes to the same record, what wins?

Last-write-wins (LWW): The write with the higher timestamp wins. Simple and widely used (DynamoDB, Cassandra default). Problem: if clocks are not synchronized

(clock skew), the "newer" timestamp might actually be from an older write. Also, LWW silently discards data.

Causality tracking with vector clocks: Each node maintains a vector clock — a list of (node, sequence_number) pairs tracking which writes from which nodes it has seen. When two writes have a causal relationship (A caused B), their vector clocks are comparable. When two writes are concurrent (neither caused the other), their vector clocks are incomparable — this is a conflict that needs resolution.

CRDTs (Conflict-free Replicated Data Types): Data structures designed to merge automatically without conflicts. Examples:

- **G-Counter:** Each node has a counter. To increment, increment only your own counter. Total = sum of all counters. Merging is max per node.
- **LWW-Element-Set:** A set where each element has a timestamp. Conflicts resolved by last-write-wins per element. Supports add and remove.
- **OR-Set (Observed-Remove Set):** Uses unique tags to distinguish adds from removes — solves the "remove element that was concurrently re-added" problem.
- **MV-Register (Multi-Value Register):** Keeps all concurrent values; surfaces the conflict to the application.

Leaderless Replication (Dynamo-Style)

No designated leader. Any node can accept writes for any data. Used by: Apache Cassandra, Riak, Amazon DynamoDB (internal architecture).

Quorum Reads and Writes

With N replicas, W required write acknowledgments, and R required read acknowledgments:

- **Strong consistency:** $W + R > N$. Example: $N=3$, $W=2$, $R=2$. Any read will overlap with at least one write recipient, guaranteeing the latest value is included.
- **High availability writes:** $W=1$, $R=N$ (or $R=1$ for eventual reads).

Cassandra's default with replication factor 3 and QUORUM consistency: $W=2$, $R=2$, $N=3$. $2+2>3$ — strong consistency guaranteed.

Sloppy Quorum and Hinted Handoff

In a traditional Dynamo-style system, if a node is unavailable, a write quorum might not be achievable, causing the write to fail. Sloppy quorum relaxes this: if the designated replica nodes are unavailable, write to any available node instead. That node stores a "hint" that this data belongs to the unavailable node and will forward it when the node recovers (hinted handoff).

Sloppy quorum improves write availability at the cost of read consistency: after a sloppy write, reading from the designated replica nodes won't return the latest value (it's on a different node). Cassandra uses sloppy quorum.

Anti-Entropy and Read Repair

Two mechanisms to bring diverged replicas back into sync:

Read repair: When a read is performed, the coordinator reads from R replicas and compares them. If they disagree (replication diverged), the coordinator writes the latest value to the stale replicas. Read repair only fixes data that is read — "cold" data that is never read can stay stale indefinitely.

Anti-entropy: A background process runs on each node. It uses Merkle trees to efficiently identify diverged data between nodes and synchronizes it. A Merkle tree is a hash tree where each leaf is the hash of a range of data, and each internal node is the hash of its children. To compare two nodes' data, compare their Merkle tree roots. If they differ, recurse down the tree to identify exactly which ranges differ — this requires only $O(\log n)$ network messages to identify all divergences.

Chapter 16: Sharding and Partitioning — Strategies and Tradeoffs

Why Shard?

A single database node has physical limits: a maximum amount of data it can store on disk, a maximum number of requests it can handle per second (CPU-bound or memory-bound), and a maximum write throughput (WAL/flush rate on a single storage device).

When your workload exceeds these limits, you must distribute the data across multiple nodes. Sharding (also called partitioning) divides the dataset into shards, each shard containing a subset of the data, hosted on a separate node (or set of nodes for replication).

Hash Sharding

How it works: For each record, compute `shard_id = hash(partition_key) % num_shards`. Route the record to the node owning that shard.

Example: User data sharded on `user_id`. `hash("user_12345") % 16 = 7` → shard 7 → node-07.

Advantages:

- Uniform distribution if the partition key has high cardinality and is uniformly distributed
- Simple routing logic
- Eliminates hotspots caused by ordered keys (timestamp-based writes all going to the "latest" shard)

Disadvantages:

- Range queries require scatter-gather: to find all users created in the last 30 days, you must query all shards (since the date is not the partition key, users in that date range are

distributed across all shards)

- Rebalancing when adding shards: changing `num_shards` from 16 to 17 changes the shard assignment of nearly all records — massive data movement required

Consistent hashing solves the rebalancing problem. Instead of `hash(key) % N`, you map both keys and nodes to a ring (0 to 2^{32}). A key is assigned to the first node clockwise on the ring. Adding a node only requires moving data from the new node's predecessor. Removing a node only requires moving that node's data to its successor. At most $1/N$ of the data moves on each topology change. Virtual nodes (many small positions per physical node) ensure even distribution.

Range Sharding

How it works: Divide the key space into ordered ranges. Each node owns a contiguous range of keys. Shard 1 owns keys A-M, shard 2 owns keys N-Z.

Advantages:

- Range queries are efficient: a query for all keys between "apple" and "banana" goes to one (or a few) shards
- Keys that are accessed together tend to be on the same shard (locality)

Disadvantages:

- Hotspots: if the application always writes to the current highest key (e.g., timestamps as keys), all writes go to the last shard — "last-writer hotspot"
- Uneven distribution: some ranges may have far more data or traffic than others

Mitigation: Salt the key with a random prefix: `shard_key = random_prefix + timestamp`. This distributes writes evenly across shards but sacrifices range scan locality. Or use a different partition key (like the sensor ID for time-series, so different sensors' data is on different shards and each shard gets range scan locality for one sensor's data).

Directory-Based Sharding

A separate lookup service (the "directory") maps each key (or key range) to a shard. Instead of computing the shard from the key, you ask the directory.

Advantages:

- Maximum flexibility: any key can be remapped to any shard without moving data
- Non-uniform distribution can be handled manually (put hot keys on dedicated shards)

Disadvantages:

- The directory is a centralized bottleneck and single point of failure
- Every read/write requires a directory lookup (add network round trip, or cache locally)
- Cache invalidation: if directory changes while clients have cached old mappings, requests go to the wrong shard

Cross-Shard Operations — The Nightmare Scenario

Sharding is painful when you need to operate on data that spans multiple shards.

Cross-shard queries: A query that needs data from multiple shards must be sent to each shard, results collected by the application, and merged/sorted in memory. No JOINS across shards. No global ORDER BY without fetching from all shards and sorting in the application.

Cross-shard transactions: Atomically updating records on different shards requires a distributed transaction protocol. Two-Phase Commit (2PC):

1. Phase 1 (Prepare): coordinator sends "prepare" to all participant shards. Each shard writes its changes to a log and responds "ready" or "abort."
2. Phase 2 (Commit): if all shards responded "ready," coordinator sends "commit" to all. If any shard sent "abort," coordinator sends "rollback" to all.

2PC is blocking: if the coordinator fails after phase 1 but before phase 2, participant shards are stuck in a "prepared" state indefinitely, holding locks. This is the "blocking

problem" of 2PC. Modern systems use Paxos/Raft-based consensus (like CockroachDB) to eliminate the coordinator as a single point of failure.

The golden rule: Design your sharding scheme to minimize cross-shard operations. Shard on the entity that is accessed most frequently together. For an e-commerce site, shard by `customer_id` — all orders, cart items, addresses, and payment methods for one customer are on the same shard, making customer-centric queries shard-local.

Hotspots and the Celebrity Problem

A hotspot is a shard that receives disproportionate traffic — a "hot" shard that becomes a bottleneck while other shards are idle.

The celebrity problem: In a social network, Justin Bieber has 200 million followers. If the timeline is sharded by `user_id`, the shard containing Justin Bieber's data must handle all read requests from 200 million followers reading his tweets. No amount of additional shards helps — his data is on one shard.

Solutions:

- **Read replicas for hot shards:** Replicate the hot shard to multiple nodes and load-balance reads across them.
 - **Application-level caching:** Cache Justin Bieber's latest tweets in Redis. Serve the cached version to all followers without hitting the DB shard.
 - **Hybrid fan-out:** Pre-compute the timelines of most users (fan-out on write) but serve celebrity tweets at read time (fan-out on read), merged at the application layer.
-

Chapter 17: Indexing — Deep Internals

B-Tree Index — The Workhorse

The B-tree (Balanced tree) is the index structure underlying almost all SQL database indexes. Unlike a binary tree (at most 2 children per node), a B-tree node can have hundreds of children, making the tree very wide and shallow. A B-tree with a branching factor of 500 and 4 levels can hold $500^4 = 62.5$ billion entries — the root and all internal nodes fit in a few pages of RAM, so most searches require only one disk read (the leaf page).

Structure: internal nodes store separator keys and pointers to child nodes. Leaf nodes store the actual index entries (key + pointer to heap row, or key + primary key for secondary indexes). All leaf nodes are at the same depth (balanced). Leaf nodes are doubly linked (for range scans — traverse leaf pages without going back up the tree).

B-tree operations:

Search: descend from root to leaf following the path where `key >= separator`.
 $O(\log n)$ disk reads, typically 2–4 for a large table.

Insert: find the leaf page where the key belongs. If the leaf has space, insert and done. If the leaf is full, split it into two half-full pages and promote the middle key to the parent. If the parent is full, split it too (cascading splits up the tree). The tree grows by splitting the root, maintaining balance.

Update: find the leaf, update in place (or delete + insert if the key changes).

Delete: find and remove the leaf entry. If the page becomes less than half full, it may be merged with a sibling or keys redistributed. (Most databases don't bother with merging on delete — they leave sparsely-populated pages and eventually reclaim space via VACUUM/reorganization.)

B+tree (variant used by all modern databases): Unlike classic B-tree (which can store data in internal nodes), B+trees only store data in leaf nodes. Internal nodes only store separator keys. This means: (a) internal nodes can have higher branching factor (more keys per internal node since no data stored), making the tree shallower; and (b) range scans are efficient (all data is in leaf nodes, which are a doubly-linked list — no need to backtrack through internal nodes).

LSM-Tree (Log-Structured Merge-Tree)

LSM-trees take the opposite approach from B-trees. Instead of updating data in place, writes are always appended to an in-memory structure (memtable) and periodically flushed to disk as immutable sorted files (SSTables). This converts random writes into sequential writes, achieving much higher write throughput.

Write path:

1. Write to WAL (for crash recovery)
2. Write to memtable (in-memory sorted tree, typically a red-black tree or skip list)
3. When memtable reaches threshold, flush to SStable on disk (sorted, compressed)

Read path:

1. Check memtable (most recent data)
2. Check most recent SSTables (may have more recent data)
3. Check older SSTables
4. Merge results (most recent version of each key wins)

The problem: As SSTables accumulate, reads require checking many files — "read amplification." Compaction periodically merges SSTables, removing old versions of keys and tombstones for deleted keys.

Compaction strategies (RocksDB):

Size-tiered compaction: group SSTables of similar size and compact them. Good write throughput, poor read performance (many levels to check).

Leveled compaction: organize SSTables into levels. Level 0 is the output of memtable flushes. Levels 1-N have a size limit (10× per level). When a level is full, it merges one SSTable with overlapping SSTables in the next level. At any given level, key ranges don't overlap — reads check at most one SSTable per level. Better reads, more write amplification (the same data is rewritten many times as it progresses through levels).

FIFO compaction: the simplest strategy — keep only the most recent SSTables, discard old ones. Good for append-only time-series data that expires by age.

Bloom filters: Before checking an SSTable for a key, consult its Bloom filter. If the Bloom filter says the key is not in the SSTable, skip the SSTable. This eliminates the disk read for the common case where the key doesn't exist in that SSTable. Bloom filters have no false negatives (if the filter says "not present," it's definitely not there) but can have false positives (the filter says "present" but the key is actually not there — rare, ~1% rate for typical configurations).

Covering Indexes and Index-Only Scans

A covering index includes all columns needed by a query, so the database can answer the query entirely from the index without accessing the main table (heap).

Example: a query that frequently runs `SELECT user_id, email FROM users WHERE status = 'active' ORDER BY created_at DESC LIMIT 100`.

Create a covering index: `CREATE INDEX idx_users_status_created_email ON users (status, created_at DESC) INCLUDE (user_id, email);`

Now the query planner can perform an index-only scan: traverse the index (filtering `status = 'active'`, reading in `created_at` order), extract `user_id` and `email` from the index itself, without touching the heap at all. Heap I/O eliminated entirely.

The `INCLUDE` clause (PostgreSQL 11+) adds columns to the leaf level of the index without including them in the sort key — they're present for retrieval but not for searching.

Partial Indexes

An index that covers only a subset of rows — those matching a WHERE clause.

```
CREATE INDEX idx_pending_orders ON orders (created_at) WHERE status = 'pending'
```

This index is much smaller than a full index on `created_at` (only pending orders, not all orders). Queries filtering on `status = 'pending'` use this index. Queries on other statuses do not benefit from it.

Partial indexes are ideal for indexing a small subset of rows that are heavily queried — like pending tasks, unprocessed events, or active sessions.

Composite Indexes and Column Order

A composite index covers multiple columns. The column order in the index determines which queries benefit from it.

Rule: a composite index can be used for queries that filter on a leading prefix of its columns. An index on `(a, b, c)` supports queries filtering on: `a`, `(a, b)`, `(a, b, c)`. It cannot efficiently support queries filtering only on `b` or `c` (must scan entire index).

Index on `(user_id, created_at)`:

- `WHERE user_id = 123` → can use index (filter on leading column)
- `WHERE user_id = 123 AND created_at > '2024-01-01'` → can use index (filter on both columns)
- `WHERE created_at > '2024-01-01'` → cannot use index efficiently (`created_at` is not the leading column)

Column order heuristic: put the equality filter columns first, then the range filter column last. The equality filter narrows the index scan to a specific prefix; the range filter then scans within that prefix.

Chapter 18: Caching — Patterns, Strategies, and Pitfalls

Why Cache?

Caching stores a subset of data in a faster-access layer (typically RAM) to reduce the number of requests that reach the slower underlying database. A database serving 100ms average query latency, fronted by a Redis cache with 99% hit rate, sees only 1% of its original request volume — an effective 100× reduction in DB load, and the 99% of requests served from cache complete in < 1ms instead of 100ms.

Cache effectiveness is measured by the hit rate: percentage of requests served from cache. A 90% hit rate means 10× DB load reduction. A 99% hit rate means 100× reduction. The difference between 90% and 99% is enormous — design to maximize hit rate.

Cache-Aside (Lazy Loading)

The most common caching pattern. The application manages the cache explicitly.

Read path:

1. Check cache for key. Cache hit → return value.
2. Cache miss → query database.
3. Store result in cache with TTL.
4. Return value.

Write path:

1. Write to database.

2. Invalidate (delete) the cache key. (On the next read, it will be refreshed from the database.)

Advantages:

- Only caches data that is actually read — efficient use of cache memory
- Cache failures are non-fatal (the application falls through to the database)
- Works for any data store

Disadvantages:

- Cache miss penalty: first request after cache invalidation always hits the database
- Potential for stale data if the cache is not invalidated correctly
- Cache stampede: if many requests miss the cache simultaneously (e.g., after a restart or after a popular key expires), they all hit the database at once — "thundering herd"

Cache stampede prevention:

- **Mutex/lock on cache miss:** Only the first request acquires a lock and populates the cache. Subsequent requests for the same key wait for the lock. Prevents simultaneous DB hits.
- **Probabilistic early expiration:** Before a key expires, probabilistically refresh it. As expiration approaches, each read has an increasing probability of triggering a refresh — spreading the refresh load over time.
- **Background refresh:** Keep the key alive (extend TTL) while a background process asynchronously refreshes its value.

Write-Through Cache

The application writes to both the cache and the database (synchronously) on every write. The cache is always up-to-date.

Advantages:

- Cache is always consistent with the database
- No stale reads after writes

Disadvantages:

- Write latency increases (must write to both cache and DB)

- Cache fills with data that may never be read (every write populates the cache, even for keys never queried)

Use when reads far outnumber writes and read latency is critical.

Write-Behind (Write-Back) Cache

The application writes to the cache only. The cache asynchronously flushes writes to the database. The database is not the first target — the cache is.

Advantages:

- Write latency is cache latency (sub-millisecond)
- Batching: multiple writes to the same key can be coalesced (only the final value is written to the DB)
- Very high write throughput

Disadvantages:

- Data loss risk: if the cache fails before flushing to the database, writes are lost
- More complex implementation
- Database may be stale while writes are in flight

Use for write-heavy workloads where small amounts of data loss are acceptable, and where the DB is the bottleneck.

Cache Invalidation — The Hardest Problem

"There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton

Cache invalidation is hard because it requires coordinating the cache and the database to stay in sync, and failures or race conditions can leave stale data in the cache indefinitely.

Time-To-Live (TTL): Every cached value expires after a configured time. Simple and always-safe — stale data self-expires. The tradeoff: users see stale data for up to TTL

seconds after a write. For slowly-changing data (product prices, user profiles), a TTL of 60–300 seconds is acceptable.

Event-driven invalidation: When a write occurs, explicitly invalidate the affected cache key(s). Guarantees near-instantaneous consistency at the cost of correctly identifying which keys to invalidate.

The pathological case: write to user's profile → invalidate `user:123:profile` → but the user's profile is also embedded in `user:123:feed`, `user:123:notifications`, etc. — you must identify and invalidate all affected cache keys. This becomes complex as the system grows.

Cache-aside with version keys: Instead of invalidating `user:123`, use `user:123:v7` (include a version number). On write, increment the version stored in a separate key (`user:123:version` → 8). Each cache read first fetches the current version, then fetches `user:123:v8`. Old version keys expire by TTL. No explicit invalidation needed — old keys become unreachable.

Distributed Cache Topology

Look-aside (client-side) cache: Each application server has a local in-memory cache (e.g., Caffeine/Guava cache in Java). Zero network overhead. But each server has its own cache — one server might have a cached value that another server doesn't. Total cache size = per-server cache size (not additive). Invalidation is per-server. Best for: truly immutable data (static config, reference data) or data where cross-server inconsistency is acceptable.

Distributed cache (Redis/Memcached): All application servers share a single distributed cache cluster. Cache misses always go to the cache cluster, never to each server independently. Total cache size = sum of all cache nodes. Invalidation is centralized. Adds one network hop per cache operation (~0.1ms to a co-located Redis). The standard pattern for any multi-server deployment.

Multi-tier cache: Local cache in front of distributed cache. Check local cache first (zero network); miss → check Redis; miss → hit database. Reduces Redis load by

~90% for hot keys. Adds complexity: local cache can be stale if another server writes to Redis. Only appropriate for data that can tolerate brief (seconds) inconsistency across servers.

CDN Caching — Caching at the Edge

Content Delivery Networks (CDNs) like Cloudfront, Akamai, and Cloudflare cache responses at globally distributed edge nodes, serving requests from the node closest to the user.

CDN caching works for any cacheable HTTP response: static assets (JS, CSS, images, videos) are the obvious case. But APIs can also be CDN-cached for responses that are the same for all users (product listings, news feeds, search results for popular queries).

Cache-Control headers control CDN behavior:

- `Cache-Control: max-age=3600` — cache for 1 hour
- `Cache-Control: s-maxage=3600, max-age=0` — cache at CDN for 1 hour, not in browser
- `Cache-Control: no-cache` — must revalidate with origin before serving
- `Vary: Accept-Language` — cache separately per language

CDN cache invalidation (purging) is supported by all major CDNs but adds operational complexity. Design APIs so that URLs are version-specific (include content hash in the URL for static assets) — then old URLs expire by TTL and new URLs are always cache misses on the first fetch.

Chapter 19: Consistency Patterns and Distributed Coordination

Distributed Transactions

Distributed transactions require atomicity across multiple nodes or services. When you transfer money between two bank accounts on different shards, both updates must commit or both must roll back.

Two-Phase Commit (2PC)

The classic protocol for distributed transactions:

Phase 1 — Prepare:

- Coordinator sends "PREPARE" to all participants (shards/services involved in the transaction).
- Each participant: write the transaction to its WAL, acquire locks, and respond "READY" or "ABORT."
- If any participant responds "ABORT," the transaction fails.

Phase 2 — Commit (or Rollback):

- If all participants responded "READY," coordinator sends "COMMIT" to all.
- Each participant commits, releases locks, and acknowledges.
- If any participant responded "ABORT," coordinator sends "ROLLBACK" to all.

The blocking problem: If the coordinator fails after sending PREPARE but before sending COMMIT/ROLLBACK, participants are stuck in the "prepared" state indefinitely, holding locks and preventing other transactions from accessing the locked data. The system is blocked until the coordinator recovers.

Three-Phase Commit (3PC): Adds a "pre-commit" phase to avoid blocking. More complex, still vulnerable to network partitions. Rarely used in practice.

Practical solution: Use Raft/Paxos for coordinator consensus, making the coordinator itself fault-tolerant. CockroachDB's distributed transaction protocol works this way.

Saga Pattern

The Saga pattern is the recommended approach for distributed transactions in microservices architecture. Instead of a single distributed ACID transaction, a saga is a sequence of local transactions, each of which publishes events or messages. If a local transaction fails, compensating transactions are executed to undo the effects of prior transactions.

Example: online order saga:

1. Create order (Order service) → publishes "OrderCreated"
2. Reserve inventory (Inventory service) → publishes "InventoryReserved"
3. Process payment (Payment service) → publishes "PaymentProcessed"
4. Ship order (Shipping service) → publishes "OrderShipped"

If step 3 (payment) fails:

- Compensate step 2: "Release reservation" (Inventory service)
- Compensate step 1: "Cancel order" (Order service)

Choreography-based saga: Services react to events from each other, no central coordinator. Simple to implement, but hard to track overall state and troubleshoot failures.

Orchestration-based saga: A central Saga Orchestrator sends commands to services and listens for events. The orchestrator knows the current state of the saga and decides what to do next. Easier to monitor and debug, but the orchestrator is a central component that must be fault-tolerant.

Idempotency is essential for sagas: Each step must be idempotent. If a compensating transaction is retried (because the first attempt's response was lost), it must produce the same result without causing errors.

Leader Election

Many distributed systems need a leader — one node that coordinates others. The challenge: how do all nodes agree on who the leader is, especially when the current leader might fail or be partitioned?

Paxos: The foundational distributed consensus algorithm. In Paxos, a proposer proposes a value, acceptors vote on proposals, and learners learn the agreed value. A value is chosen when a majority of acceptors agree on it. Paxos is notoriously difficult to implement correctly (edge cases in leader failure, network partitions, and message reordering are numerous).

Raft: A consensus algorithm designed to be more understandable than Paxos. Raft explicitly separates: leader election, log replication, and safety. Leader election: each node starts as a follower. If a follower doesn't hear from a leader within a timeout (election timeout, randomized to avoid split votes), it starts an election, incrementing its term and requesting votes. A node wins if it gets a majority of votes. Log replication: the leader accepts client requests, appends them to its log, replicates to followers (AppendEntries RPC), and commits entries once a majority have confirmed.

Zookeeper and etcd provide leader election as a service. Distributed applications use ZooKeeper/etcd as an external coordination service: candidates create ephemeral nodes (ZooKeeper) or keys with leases (etcd). The candidate that successfully creates the node becomes the leader. When the leader process dies, its ZooKeeper session expires (ephemeral node deleted), and other candidates race to create the node.

Distributed Locking

When multiple processes might concurrently modify the same resource, distributed locking ensures mutual exclusion. Naively, use Redis:

```
SET lock:resource_123 {client_id} NX EX 30
```

NX = set only if not exists (acquire lock only if no one holds it). **EX 30** = expire in 30 seconds (safety net if the lock holder crashes without releasing).

The Redlock problem: Martin Kleppmann famously critiqued Redlock (a multi-node Redis distributed lock algorithm) for having correctness issues under network partitions and clock skew. For situations where correctness is critical (double-purchase prevention, financial operations), use a database row-level lock or a consensus-based lock service (etcd, ZooKeeper) rather than Redis.

Fencing tokens: A safer pattern. The lock service issues a monotonically increasing token with each lock grant. The token is passed to every operation the lock holder performs. The resource checks that each token is greater than the last seen token — if an expired lock holder tries to perform an operation after losing the lock (e.g., due to a long GC pause), its old token will be rejected.

Chapter 20: System Design Patterns Per Use Case

Pattern 1: Social Media Feed (Twitter/Instagram)

Requirements

- Users follow other users
- Users post content (tweets, posts)
- Users view a feed of content from people they follow, ordered by time
- Scale: hundreds of millions of users, millions of posts per day

The Fan-Out Problem

The core challenge: when user A (with N followers) posts, how do all N followers see the post in their feed?

Fan-out on write (push model): When a post is created, write a copy to the feed of every follower. Feed reads are fast (pre-computed). Feed writes are slow and expensive for high-follower accounts. Suitable for: users with low follower counts.

Fan-out on read (pull model): When a user views their feed, query the "posts" table for each account they follow, merge and sort results. No write-time work. Read is slow (N queries for N followees, then merge). Suitable for: accounts with millions of followers (celebrities) to avoid write amplification.

Hybrid model (the production solution): Fan-out on write for regular users; fan-out on read for celebrities. At read time, merge the pre-computed feed with a real-time query of celebrity accounts the user follows.

Data Model

Posts table (PostgreSQL): `post_id`, `user_id`, `content`, `media_url` (S3 key), `created_at`. Indexed by (`user_id`, `created_at` DESC) for "all posts by user" queries.

Follow graph (Graph DB or denormalized): For casual follower traversal (who follows whom), a simple relational table works: `followers(follower_id, followee_id, created_at)`. For complex social graph analytics, a graph database is appropriate.

Feed table (Cassandra): Pre-computed feed per user. Partition key: `user_id`. Clustering key: `post_time` DESC, `post_id`. Wide rows — a user's timeline is one partition, ordered by time.

Cache (Redis): The most recent 100 posts of a user's feed cached in a sorted set (score = timestamp, member = serialized post). Cache hit on feed view is the norm; Cassandra only queried for pagination beyond page 1.

Media storage (S3 + CDN): Videos and images stored in S3. CloudFront CDN for global distribution. Thumbnails generated asynchronously by a Lambda on S3 upload event.

Pattern 2: Ride-Sharing (Uber/Lyft)

Requirements

- Riders request rides; drivers accept
- Match nearby drivers to riders based on location
- Track driver location in real-time
- Route optimization

Geospatial Indexing

The core challenge: "find all drivers within 5km of this rider's location."

Geohash: A geohash encodes a (latitude, longitude) pair into a compact string (e.g., "9q8yy9mf"). The string length determines precision: 6 characters $\approx 1.2\text{km} \times 0.6\text{km}$ cell; 7 characters $\approx 153\text{m} \times 153\text{m}$. Adjacent cells have similar prefixes (with edge cases at geohash boundaries). Store driver locations with their geohash. Query: find drivers in the same geohash cell (and adjacent cells for boundary coverage).

PostGIS + PostgreSQL: The gold standard for geospatial queries.

`ST_DWithin(driver_location, rider_location, 5000)` finds all drivers within 5km. Backed by an R-tree index (GiST index in PostgreSQL). Very expressive — arbitrary polygon queries, distance calculations, routing.

Redis Geospatial: `GEOADD drivers {lng} {lat} {driver_id}` and `GEOADIUS drivers {lng} {lat} 5 km` for real-time driver location. Sub-millisecond for location updates and nearby searches. Appropriate for the real-time dispatch component (writes every few seconds per driver), while PostGIS handles complex geospatial analytics.

Driver Location Pipeline

Driver app sends GPS coordinate every 4 seconds → API Gateway → Kafka topic (driver_locations) → Stream processor (Flink/Spark Streaming) → Redis (for real-time dispatch lookup) + PostgreSQL (for trip reconstruction/analytics).

Never write driver GPS directly to PostgreSQL at $500,000 \text{ drivers} \times 1 \text{ update/4 seconds} = 125,000 \text{ writes/second}$. Kafka absorbs the burst; downstream writers batch to PostgreSQL.

Pattern 3: Web-Scale URL Shortener (bit.ly)

Requirements

- Convert long URLs to short codes
- Redirect short codes to long URLs
- Handle billions of redirects per day with low latency
- Optional: analytics (click counts per URL, geo/device breakdown)

Core Architecture

Short code generation:

- Option 1: Hash the long URL (MD5/SHA256), take first 6 characters of base62-encoded hash. Problem: collisions require checking the database.
- Option 2: Auto-increment ID from a database, base62-encode it. `12345 → "dnh"`. No collisions. Problem: predictable IDs.
- Option 3: Pre-generate random 6-character base62 codes in batches; store in a "codes" table. Atomic `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` to claim a code.

Database:

- **Writes:** PostgreSQL with `(short_code, long_url, created_at, user_id)`. Primary key on short_code (B-tree, supports $O(\log n)$ lookup by short_code).
- **Reads:** Redis cache. `GET short:dnh3k2 → long_url`. Cache hit rate >99.9% for popular links. TTL varies: popular links cached indefinitely; unpopular links TTL 24h.

- **Cache warming:** On new short link creation, immediately write to Redis. On cache miss (unpopular/expired), fetch from PostgreSQL and re-cache.

Analytics:

- Don't write analytics synchronously on each redirect (would slow redirects).
- Each redirect event → Kafka (`redirect_events` topic) → Flink/Spark streaming aggregation → ClickHouse (analytics warehouse) for dashboards.
- Pre-aggregate click counts in ClickHouse: hourly roll-ups, geo/device breakdown.

Pattern 4: E-Commerce Product Catalog and Search

Requirements

- Browse products by category, brand, price range
- Full-text product search with relevance ranking
- Faceted filtering (filter by multiple attributes simultaneously)
- Product detail pages

Data Model Split

PostgreSQL (source of truth): Products table with core attributes (id, name, price, brand_id, category_id, inventory_count, created_at). Transactions for inventory updates and orders. Normalized — product attributes stored in EAV table for variable attributes, or JSONB column.

Elasticsearch (search and browse): All product data denormalized into Elasticsearch documents. Indexed for full-text search on name/description, keyword filters on brand/category, numeric range on price, and aggregations for facet counts. Synced from PostgreSQL via CDC (Change Data Capture using Debezium) or application dual-write.

S3 (product images): Product images stored in S3. CloudFront CDN for fast global delivery. Thumbnails at multiple resolutions pre-generated by Lambda.

Redis (hot product cache): Top 10,000 most-viewed product pages cached as serialized JSON. Cache-aside with 10-minute TTL. Eliminates 95% of product detail page DB reads.

Pattern 5: Real-Time Chat (WhatsApp/Slack)

Requirements

- Send messages to individuals and groups
- Real-time delivery (< 1 second)
- Message persistence and history
- Online presence (who is online)

Message Storage

Cassandra for messages: `(conversation_id, message_time, message_id) → (sender_id, content, message_type)`. Partition key = `conversation_id` — all messages in a conversation in one partition, ordered by time. Fast reads for message history. High write throughput for concurrent conversations. Append-only (messages aren't usually edited or deleted).

Real-time delivery: WebSocket connections from each client to a WebSocket server. When a message is sent, the server writes it to Cassandra AND pushes it in real-time to the recipient's WebSocket connection. If the recipient is offline, the push fails gracefully — they'll fetch history from Cassandra when they reconnect.

Presence (who is online): Redis `SETEX presence:user:123 60 "online"` — user pings the server every 30 seconds to renew. Check `EXISTS presence:user:123` to test if online. Expiry provides automatic offline detection without explicit logout.

Message queue for offline delivery: If recipient is offline, write to an SQS/Kafka queue. When they reconnect, their client fetches recent messages from Cassandra and subscribes to pending messages in the queue.

Chapter 21: Numbers Every Engineer Must Know

These numbers represent order-of-magnitude estimates that inform back-of-envelope calculations in system design interviews. Memorize these not as rote facts but as intuitions about the relative cost of operations.

Latency Numbers (2024 Hardware)

OPERATION	LATENCY	NOTES
L1 cache reference	~1 ns	Fastest possible memory access
Branch misprediction	~3 ns	CPU pipeline stall
L2 cache reference	~4 ns	Still extremely fast
Mutex lock/unlock	~25 ns	Even locking is cheap
L3 cache reference	~40 ns	Shared across cores
Main memory (RAM) reference	~100 ns	100× slower than L1
Compress 1 KB with Snappy	~3 μs	Fast compression
Read 4 KB from NVMe SSD	~20–100 μs	Flash storage is fast

OPERATION	LATENCY	NOTES
Read 4 KB from SATA SSD	~100 μ s	Slower flash interface
Redis GET (local network)	~100–500 μ s	In-memory, one network hop
PostgreSQL indexed query	~1–10 ms	Depends on buffer cache hits
Read 1 MB sequentially from NVMe SSD	~200 μ s	Sequential reads are fast
Round trip within same datacenter	~0.5 ms	Intra-DC LAN
Read 1 MB sequentially from HDD	~2–5 ms	Seek + rotational latency
HDD seek	~5–10 ms	Mechanical movement
Round trip between US datacenters	~30–50 ms	Cross-country
Round trip US $\rightarrow$ Europe	~70–100 ms	Transatlantic
Round trip US $\rightarrow$ Asia	~100–200 ms	Transpacific

Throughput and Capacity Numbers

SYSTEM	THROUGHPUT	NOTES
Single NVMe SSD sequential read	~3–7 GB/s	PCIe 4.0
Single NVMe SSD sequential write	~2–5 GB/s	
Single NVMe SSD random read	~700K–1M IOPS	4KB blocks
10 Gbps network link	1.25 GB/s	Full duplex

SYSTEM	THROUGHPUT	NOTES
Redis	100K–1M ops/sec	Single node, pipelined
Nginx	~50K req/sec	Single server, static files
PostgreSQL (OLTP)	5K–50K queries/sec	Depends heavily on query
Kafka	1M+ msgs/sec	Per-broker, sequential writes
Cassandra write	100K+ writes/sec	Per-node, commodity hardware
ClickHouse aggregation	1B+ rows/sec	Single node, vectorized

Storage Sizes

ITEM	SIZE
ASCII character	1 byte
UTF-8 character (English)	1 byte
UUID (stored as string)	36 bytes
UUID (stored as binary)	16 bytes
Tweet (text only)	~200–300 bytes
User record (typical)	~500 bytes–2 KB
HD image (JPEG)	~1–5 MB
4K image (JPEG)	~5–15 MB
1 minute of MP3 audio	~1 MB

ITEM	SIZE
1 minute of 1080p video	~100–300 MB
1 minute of 4K video	~500 MB–1 GB

Back-of-Envelope Calculation Framework

The formula for any capacity estimate:

```
Daily data = (requests per day) × (data per request)
QPS (Queries Per Second) = daily requests / 86,400 seconds
Peak QPS = QPS × 2–10x (depending on traffic pattern)
Storage per year = daily data × 365
```

Quick approximations:

- 1 million requests/day = ~12 requests/second
- 100 million requests/day = ~1,200 requests/second (1.2K QPS)
- 1 billion requests/day = ~12,000 requests/second (12K QPS)
- 86,400 seconds per day (≈ 100K for rough calculations)

Twitter estimation example:

- 500 million tweets/day = ~6,000 tweets/second
- Peak = ~30,000 tweets/second
- Average tweet = 200 bytes
- Daily write = 500M × 200 bytes = 100 GB/day
- Yearly write = 36.5 TB
- 5 years of tweets: ~180 TB (uncompressed, ~36 TB with 5:1 compression)
- 500 million users × 500 bytes average profile = 250 GB for all user profiles — fits in RAM on a single large server

Chapter 22: Interview Strategy and Common Mistakes

The System Design Interview Framework

A system design interview is not a test of whether you know the "right answer." There is rarely a single right answer. It is a test of your ability to:

1. Clarify requirements without being told what to do
2. Make reasoned decisions with explicit tradeoffs
3. Identify bottlenecks and address them before being prompted
4. Communicate clearly about complex technical topics

Step 1: Clarify Requirements (5 minutes)

Never start designing before you understand what you're building. Ask:

Functional requirements: What are the core features? (Not every feature — identify the top 3–4 that define the system's purpose.)

Scale requirements: How many users? How many requests per second? How much data? Knowing whether the system must handle 10K or 10M users completely changes the design.

Consistency requirements: Does the system require strong consistency or is eventual consistency acceptable? Are there financial or safety implications to stale reads?

Latency requirements: What is the acceptable response time? Is this an internal tool (seconds acceptable) or a user-facing feature (100ms required)?

Availability requirements: What is the uptime SLA? 99.9% (8.7 hours/year downtime) vs 99.999% (5 minutes/year downtime) implies very different architectures.

Step 2: High-Level Design (10 minutes)

Sketch the major components:

- Clients (web, mobile)
- API gateway / load balancer
- Application servers
- Databases
- Caches
- Message queues
- CDN / storage

Identify the critical data flows for the top 2–3 user stories. Walk through each flow end-to-end. Show you understand what happens at each step.

Step 3: Database Design (10 minutes)

This is where most interviews go deep. For each entity in the system:

- What database type? Why?
- What is the schema / data model?
- What indexes are needed?
- How does it scale?

State your access patterns explicitly. "Users will query orders by user_id and by created_at. So I'll create a composite index on (user_id, created_at DESC)." This shows you're thinking about performance, not just data modeling.

Step 4: Scale and Bottlenecks (10 minutes)

Take your design and stress-test it:

- What happens when traffic is 10× what you designed for?
- What is the first bottleneck? (Usually the database write path)
- How do you scale reads? (Read replicas, caching)
- How do you scale writes? (Sharding, message queues, denormalization)
- What happens if a component fails? (Replication, fallback, graceful degradation)

Step 5: Deep Dive (10 minutes)

The interviewer will ask you to elaborate on one component. Be ready to go very deep on any component — the cache invalidation strategy, the sharding approach, the consistency model, the specific database choice.

Common Mistakes

Jumping to solutions before clarifying requirements. Many candidates start drawing boxes immediately. This leads to designing the wrong system. Always clarify first.

Choosing a database because it's popular, not because it's appropriate. "I'll use MongoDB because it's flexible" — flexible for what? What are the actual access patterns? A document database for a financial system with complex transactions is a poor choice regardless of MongoDB's popularity.

Ignoring consistency requirements. "I'll use Cassandra for everything" — Cassandra is AP/BASE by default. Is that acceptable for your use case? If a user's account balance might briefly show stale data, could that be exploited?

Over-engineering. Starting with a distributed database, 5 Kafka topics, and a microservices architecture for a system that serves 1,000 users. A single PostgreSQL instance on a \$100/month server handles more traffic than most applications will ever generate. Premature complexity is the enemy of reliability.

Under-engineering. Designing a system for 100 users when the question specifies 100 million. Always match your architecture to the stated scale.

Not knowing when to denormalize. "I'll normalize everything into 3NF" — but the most common query requires joining 8 tables, and it runs on every page load. In high-traffic systems, strategic denormalization (pre-joining frequently co-read data) is not a compromise — it is a necessity.

Forgetting failure modes. What happens if Redis is down? Does the system fail (Redis is in the critical path) or degrade gracefully (fall through to the database)? Design for failures; the interviewer will probe this.

Not mentioning tradeoffs. Every design decision has a tradeoff. "I chose Cassandra for high write throughput, at the cost of no ad-hoc queries — all access patterns must be known in advance and designed into the schema." Explicitly stating the tradeoff shows senior-level thinking.

Ignoring operations. How do you deploy schema changes? How do you perform a major version upgrade? How do you restore from a backup? Production database operations are a significant part of the real engineering cost.

The Golden Rules

1. **Start simple, add complexity only when the simple solution demonstrably doesn't fit.** A single PostgreSQL instance is not naive — it is the correct solution for most systems. Add caching, sharding, and NoSQL only when you have specific reasons.
2. **Separate hot path from cold path.** The user-facing real-time path must be fast and reliable. Analytics, reporting, and non-critical processing go on the cold path (async, via queues).
3. **Never have a single point of failure.** Every component in the critical path must have a redundant counterpart. Load balancers in pairs; database primaries with replicas and automatic failover; caches with replica sets.
4. **Design for the read/write ratio.** If reads are 100× writes, optimize for reads (indexes, caching, read replicas). If writes are the bottleneck, optimize for writes (message queues, async processing, write-optimized storage).
5. **The database is almost always the bottleneck.** When a system has performance problems at scale, it is almost always the database. Scale stateless application servers first (trivially horizontal); then add caching; then add read replicas; then consider sharding. Each step adds complexity — do not jump ahead.

6. **Be honest about what you don't know.** "I know Cassandra handles this use case well, but I'm less familiar with its data modeling nuances — let me sketch the general approach and we can discuss the specifics." This is infinitely better than confidently stating something incorrect.

Summary: Database Selection Decision Tree

```
Is data strongly relational with complex queries and ACID required?
├─ YES: PostgreSQL (or MySQL for simple workloads)
│   └─ Outgrow a single node? → CockroachDB or read replicas + careful sharding
│   └─ Need global distribution + strong consistency? → Google Spanner
└─ NO: What is the primary access pattern?
    └─ Simple key-value lookups + extreme performance? → Redis / DynamoDB
    └─ Hierarchical/variable-schema documents? → MongoDB / Firestore
    └─ Massive write throughput at scale? → Cassandra / ScyllaDB
    └─ Multi-hop graph traversal? → Neo4j / Amazon Neptune
    └─ Time-ordered append-only measurements? → InfluxDB / Prometheus / Timescale
    └─ Full-text search + relevance ranking? → Elasticsearch (secondary index)
    └─ Analytical queries over large datasets? → Snowflake / ClickHouse / BigQuery
    └─ Semantic similarity / vector search? → Pinecone / Qdrant / pgvector
    └─ Binary files / media / backups? → Amazon S3 / GCS
```

This document covers the essential database knowledge for system design at the senior engineering level. The depth here represents what separates candidates who can choose a database from those who can design a system that will perform correctly under real-world conditions at scale.